
Learning More from Less: Unlocking Internal Representations for Benchmark Compression

Yueqi Zhang^{*1} Jin Hu^{*1} Shaoxiong Feng² Peiwen Yuan¹ Xinglin Wang¹ Yiwei Li¹ Jiayi Shi¹ Chuyi Tan¹
Ji Zhang¹ Boyuan Pan² Yao Hu² Kan Li¹

Abstract

The prohibitive cost of evaluating Large Language Models (LLMs) necessitates efficient alternatives to full-scale benchmarking. Prevalent approaches address this by identifying a small coreset of items to approximate full-benchmark performance. However, existing methods must estimate a reliable item profile from response patterns across many source models, which becomes statistically unstable when the source pool is small. This dependency is particularly limiting for newly released benchmarks with minimal historical evaluation data. We argue that discrete correctness labels are a lossy view of the model’s decision process and fail to capture information encoded in hidden states. To address this, we introduce REPCORE, which aligns heterogeneous hidden states into a unified latent space to construct representative coresets. Using these subsets for performance extrapolation, REPCORE achieves precise estimation accuracy with as few as ten source models. Experiments on five benchmarks and over 200 models show consistent gains over output-based baselines in ranking correlation and estimation accuracy. Spectral analysis further indicates that the aligned representations contain separable components reflecting broad response tendencies and task-specific reasoning patterns.

1. Introduction

Large Language Models (LLMs) have demonstrated strong capabilities across diverse domains and reshaped modern AI research (Achiam et al., 2023; Grattafiori et al., 2024;

^{*}Equal contribution ¹School of Computer Science, Beijing Institute of Technology ²Xiaohongshu Inc. Correspondence to: Kan Li <likan@bit.edu.cn>, Boyuan Pan <pan-boyuan@xiaohongshu.com>.

Yang et al., 2025; Marjanović et al., 2025). Benchmark suites have therefore expanded in size, often reaching tens of thousands of items, as the community continually adds new tasks and test cases while preserving comparability over time. However, full-scale evaluation at this scale incurs substantial computational and financial costs, making frequent re-evaluation impractical during rapid model iteration (Liang et al., 2023; Perlitz et al., 2024). Consequently, recent work on *benchmark compression* selects a small coreset and evaluates models on this subset to approximate full-benchmark scores and rankings (Vivek et al., 2024; Maia Polo et al., 2024; Yuan et al., 2025).

These methods generally follow a similar pipeline. They first collect output-level responses from many source models on the full benchmark, forming a model-by-item matrix of correctness or confidence. Each item is then represented by its response vector across models or by low-dimensional factors estimated from this matrix, for example through Item Response Theory, and these representations support both coreset selection and score extrapolation. This pipeline implicitly assumes broad source coverage. When only a handful of source models are available, as is often the case for newly released or long-tail benchmarks with limited historical evaluations, these output-derived item representations are estimated unreliably. As a result, the geometric structure of the item space collapses and many distinct items become indistinguishable, as shown in Figure 1, leading to less representative coresets and larger extrapolation errors.

Motivated by this limitation, we investigate the internal representations produced during inference. Unlike discrete output labels, hidden states provide a high-dimensional and continuous view of the computation that generates an answer. Prior work has shown that these states encode rich internal features of uncertainty, problem difficulty, and task-dependent reasoning patterns (Azaria & Mitchell, 2023; Zou et al., 2023; Zhang et al., 2025; Orgad et al., 2025; Wang et al., 2025a). We hypothesize that hidden states offer a dense signal for characterizing items, but exploiting this signal requires aligning heterogeneous hidden spaces across architectures. To this end, we introduce REPCORE, a framework that aligns heterogeneous hidden states into a unified

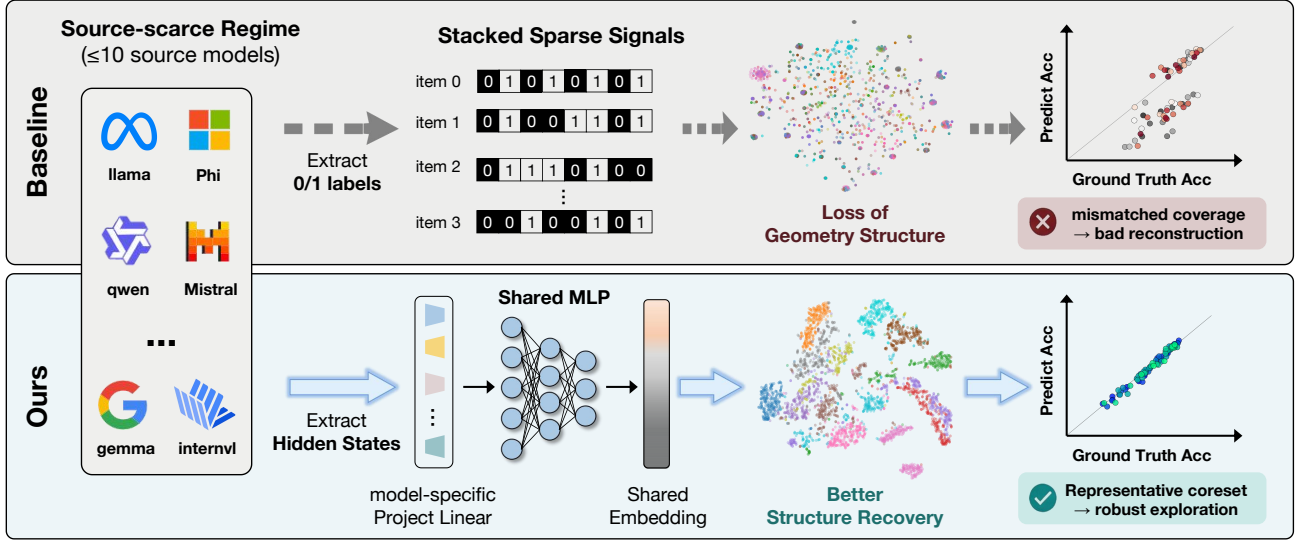


Figure 1. Comparison of item representation paradigms in source-scarce regimes. Top: Output-based methods rely on sparse 0/1 signals that fail to preserve the geometric structure of the item space. Bottom: REPCORE aligns heterogeneous hidden states into a unified latent space to recover fine-grained item structures for robust coreset selection.

latent space to recover fine-grained item structures.

REPCORE utilizes a shared projection mechanism to map model-specific hidden states into aligned embeddings, recovering the geometric structure of the item space. As shown in the bottom panel of Figure 1, aggregating these embeddings across source models reveals structured clusters associated with distinct reasoning patterns. Within this aggregated space, we employ a clustering-based strategy to select a diverse set of anchor items that serve as the training basis for a lightweight linear regressor. By extrapolating the target model’s performance from these anchors, our approach yields accurate full-benchmark estimates and remains effective even with as few as ten source models.

We conduct experiments on five benchmarks spanning text-only and multimodal settings, using over two hundred models that include text-only and multimodal variants and cover both dense and mixture-of-experts architectures. Under matched coreset sizes with ten source models, our method consistently improves full-benchmark estimation accuracy and ranking consistency over competitive output-based baselines. Our contributions are summarized as follows, with related work deferred to Appendix A:

- We formalize benchmark compression under source scarcity and show that sparse output-level signals distort the geometry of the item space, degrading both coreset selection and performance extrapolation.
- We propose REPCORE, which maps model-specific hidden states into aligned embeddings and uses the resulting space to enable robust coreset selection and accurate extrapolation with few source models.

- We provide empirical and structural evidence supporting internal representations. Our results demonstrate that the aligned embeddings contain separable components that reflect broad response tendencies and task-specific reasoning patterns.

2. RepCore Approach

We propose REPCORE, a framework designed to achieve high-fidelity benchmark compression in source-scarce regimes. As illustrated in Figure 2, the pipeline operates in three consecutive phases. First, we learn aligned representations from model hidden states to recover the benchmark’s geometric structure (§2.2). Second, we perform consensus clustering in this latent space to construct a representative coreset (§2.3). Finally, we employ a lightweight and robust extrapolator to map local coreset evaluations to full-benchmark performance estimations (§2.4).

2.1. Problem Setup

Let $\mathcal{I}$ denote the comprehensive set of items in a benchmark. We consider a set of available source models $\mathcal{S}$ and a set of unseen target models $\mathcal{T}$, operating specifically within a source-scarce regime where $|\mathcal{S}|$ is strictly limited. We adopt the standard *free-generation then binary-scoring* paradigm, where each source model $m \in \mathcal{S}$ generates a response for item $i \in \mathcal{I}$ that is deterministically parsed to yield a binary correctness label $y_{m,i} \in \{0, 1\}$. Concurrently, we extract the hidden state $h_{m,i}$ associated with the generation. Our objective is to learn item representations that capture the relational structure of the benchmark, thereby guiding the

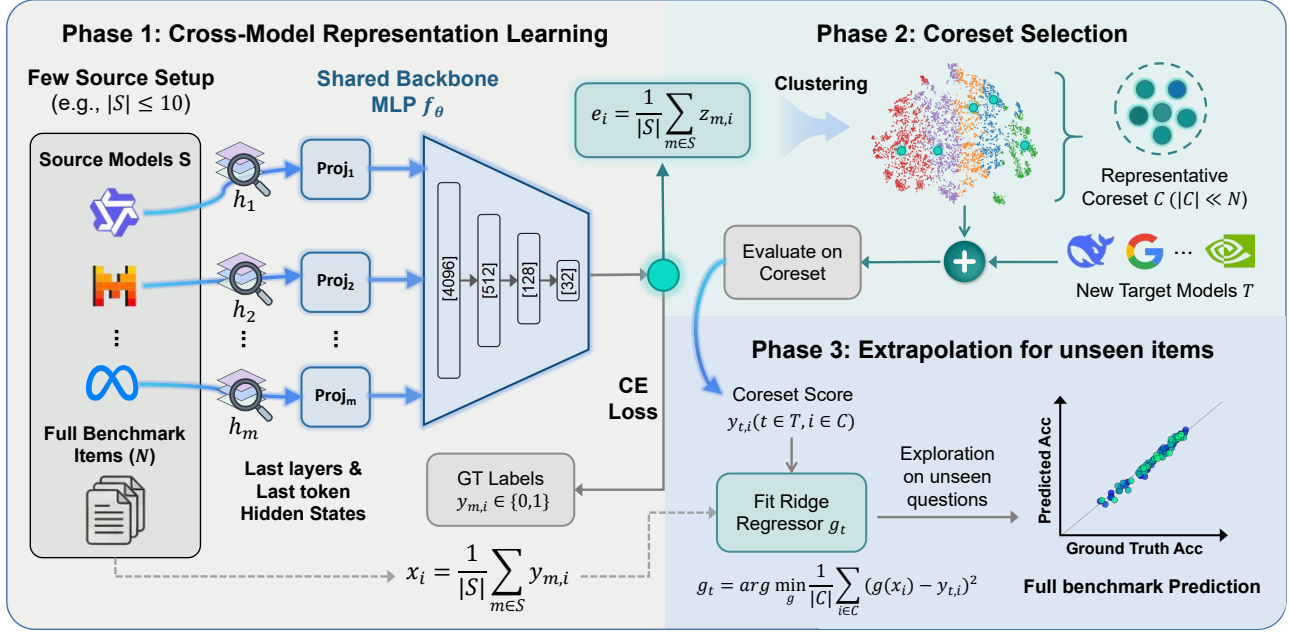


Figure 2. Overview of the REPCORE framework. The pipeline proceeds in three phases: aligning heterogeneous hidden states into a unified latent space via model-specific projections and a shared MLP, selecting representative anchor items through consensus clustering, and extrapolating full-benchmark performance using a lightweight regressor.

selection of a minimal coreset $\mathcal{C} \subset \mathcal{I}$. By evaluating target models exclusively on $\mathcal{C}$, we aim to accurately estimate their performance on the full set $\mathcal{I}$.

2.2. Cross-Model Representation Learning

Under the source-scarce regime (specifically $|\mathcal{S}| \leq 10$), the response matrix Y contains only $|\mathcal{S}|$ rows and provides a highly limited view of inter-item relations. We therefore use hidden states as dense item-level signals to recover the benchmark’s latent relational structure.

Hidden State Extraction For each model-item pair (m, i) , we extract the final layer’s hidden state of the final output token, denoted as $h_{m,i}$. Validated by recent interpretability studies (Burns et al., 2023; Wang et al., 2025c; Orgad et al., 2025; Gekhman et al., 2025), these states encode the reasoning trajectory and decision signals, providing a high-fidelity information source superior to discrete correctness labels.

Cross-Model Alignment and Supervision Source models exhibit significant architectural heterogeneity, possessing distinct hidden dimensions and feature spaces. To obtain comparable representations across models and reduce mismatch caused by heterogeneous hidden spaces, we first apply an independent model-specific linear projection layer Proj_m to each source model. For a source model m with native hidden dimension d_m , this projection maps the hidden state into a shared intermediate space, i.e.,

$\text{Proj}_m : \mathbb{R}^{d_m} \rightarrow \mathbb{R}^{4096}$. The projected features are then processed by a source-agnostic MLP f_θ shared across all source models, with layer widths $4096 \rightarrow 512 \rightarrow 128 \rightarrow 32$ (Appendix C.7 further examines alternative MLP widths and depths). This pipeline acts as an information bottleneck, progressively compressing the high-dimensional input into an aligned representation:

$$z_{m,i} = f_\theta(\text{Proj}_m(h_{m,i})) \in \mathbb{R}^{d_z} \quad (1)$$

We refer to $z_{m,i}$ as the aligned embedding of item i under source model m . The dimension d_z determines the bottleneck capacity. We set $d_z = 32$ for all subsequent experiments, as we empirically observe that for most models, the effective rank of the aligned embedding matrix (capturing 99% variance) remains below 20. Thus, 32 dimensions provide ample capacity to maintain representational richness while ensuring computational efficiency.

To explicitly extract signals directly tied to the final evaluation outcome, we apply a linear classifier g_ϕ to $z_{m,i}$ and optimize the entire network using Cross-Entropy loss to predict the ground-truth correctness $y_{m,i}$:

$$\mathcal{L} = \sum_{m \in \mathcal{S}} \sum_{i \in \mathcal{I}_{\text{train}}} \text{CE}(g_\phi(z_{m,i}), y_{m,i}) \quad (2)$$

We define $\mathcal{I}_{\text{train}}$ by randomly partitioning the target benchmark $\mathcal{I}$ into training, validation, and test splits with proportions of 70%, 10%, and 20%, respectively. This process treats the optimization as a representation learning task on the target distribution. Empirically, the classifier achieves high Area Under the Curve (AUC) scores peaking at ap-

proximately 0.9 on held-out test splits, confirming that the bottleneck features successfully distill validity signals. In this design, the classifier g_ϕ serves only as a training time supervision head, helping steer the shared latent space toward evaluation-relevant signals while reducing architecture specific variation. For the subsequent clustering phase, we utilize the MLP output $z_{m,i}$, as this aligned representation preserves richer information about the intrinsic relational structure of the full benchmark.

2.3. Coreset Selection via Consensus Clustering

To synthesize a stable structural prior from the aligned representations, we compute the consensus embedding e_i by averaging $z_{m,i}$ across all source models. We use this consensus embedding as the item representation for coreset selection. Subsequently, we apply L_2 -normalization to project the aggregated vectors onto a unit hypersphere:

$$\tilde{e}_i = \frac{e_i}{\|e_i\|_2}, \quad \text{where} \quad e_i = \frac{1}{|\mathcal{S}|} \sum_{m \in \mathcal{S}} z_{m,i} \quad (3)$$

This normalization emphasizes directional agreement and captures the geometric structure of the item space independent of vector magnitude.

Leveraging these normalized consensus embeddings, we partition the item space into K disjoint clusters $\{\mathcal{G}_k\}_{k=1}^K$ based on directional similarity. Specifically, we optimize the partition to maximize the within-cluster cosine similarity:

$$\{\mathcal{G}_k\}_{k=1}^K \leftarrow \arg \max_{\{\mathcal{G}_k\}} \sum_{k=1}^K \sum_{i \in \mathcal{G}_k} \cos(\tilde{e}_i, \mu_k) \quad (4)$$

where μ_k denotes the centroid of the k -th cluster. We identify the anchor c_k as the item maximizing the cosine similarity to the centroid:

$$\mathcal{C} = \{c_k\}_{k=1}^K \quad \text{s.t.} \quad c_k = \arg \max_{i \in \mathcal{G}_k} \cos(\tilde{e}_i, \mu_k) \quad (5)$$

The resulting coreset $\mathcal{C}$ effectively preserves the global relational structure of the benchmark while acting as a robust basis for the subsequent extrapolation phase.

2.4. Extrapolation from Coreset to Full Benchmark

To evaluate an unseen target model $t \in \mathcal{T}$, we acquire its binary scores $\{y_{t,i}\}_{i \in \mathcal{C}}$ on the selected coreset. We then employ a Ridge regression model g_t to extrapolate these local observations to the full benchmark.

Given the strictly limited coreset size, directly fitting target-specific regressors on full source-response vectors can lead to high-variance estimates. To improve robustness, we instead construct a scalar feature x_i for each item by averaging correctness across all source models:

$$x_i = \frac{1}{|\mathcal{S}|} \sum_{m \in \mathcal{S}} y_{m,i} \in [0, 1], \quad \forall i \in \mathcal{I} \quad (6)$$

This averaged score serves as a stable anchor for each item in the source-scarce regime, while target-specific variation is captured by fitting a separate regressor g_t to each model’s responses on the selected coreset. Appendix C.8 compares this scalar feature with an alternative based on the concatenated source response vector and shows that the averaged feature indeed provides more reliable ranking recovery under source scarcity.

We fit the regressor g_t by minimizing the squared error on the coreset:

$$g_t = \arg \min_g \frac{1}{|\mathcal{C}|} \sum_{i \in \mathcal{C}} (g(x_i) - y_{t,i})^2 \quad (7)$$

The optimized regressor yields predictions $\hat{s}_{t,i} = g_t(x_i)$ for each unobserved item $i \in \mathcal{I} \setminus \mathcal{C}$, providing a robust linear approximation for the unseen items.

Finally, the global accuracy is derived by aggregating the ground-truth performance on $\mathcal{C}$ with the predicted scores for the remaining items in $\mathcal{I} \setminus \mathcal{C}$. This formulation recovers both absolute performance and relative rankings while maintaining minimal computational overhead:

$$\widehat{\text{Acc}}(t) = \frac{1}{|\mathcal{I}|} \left(\sum_{i \in \mathcal{C}} y_{t,i} + \sum_{i \in \mathcal{I} \setminus \mathcal{C}} \hat{s}_{t,i} \right) \quad (8)$$

3. Main Experiments

3.1. Experimental Setup

Benchmarks and Model Pools We evaluate REPCORE on five benchmarks spanning text-only and multimodal domains. ARC Challenge (Clark et al., 2018) contains 1,172 science QA items evaluated by 96 models. BIG-Bench Hard (BBH) (Suzgun et al., 2023) includes 6,511 challenging reasoning items across 101 models. GSM8K (Cobbe et al., 2021) consists of 1,319 grade-school math word problems with 90 models. MMLU-Pro (Wang et al., 2024) contains 12,032 multi-domain questions evaluated by 91 models. SEED-Bench-2-Plus (Li et al., 2024) is a multimodal benchmark comprising 2,277 items and 93 models. A comprehensive list of models for each benchmark is provided in Appendix D.

Implementation Details We conduct all evaluations in the source-scarce regime, defined by $|\mathcal{S}| = 10$. For each benchmark, we randomly sample 10 models to form the source set $\mathcal{S}$ and designate the remaining models as the target set $\mathcal{T}$. We report results averaged over 10 independent source/target splits. All model generations utilize greedy decoding to ensure reproducibility. To ensure fair comparison, all methods operate on identical splits and coreset sizes. Since REPCORE additionally records hidden states during source-model inference, we quantify the resulting wall-clock cost and storage overhead in Appendix C.5.

Table 1. Main results. We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$) on target models. The budget K denotes the number of items in the coreset. **Bold** and underlined values denote the best and second-best results, respectively.

Benchmark	Method	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$	
		$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
BBH	RANDOM	0.668	0.122	0.793	0.087	<u>0.852</u>	0.069	<u>0.884</u>	0.061	<u>0.905</u>	0.054
	ANCHORPOINTS	<u>0.701</u>	0.127	<u>0.799</u>	0.097	<u>0.835</u>	0.087	<u>0.854</u>	0.081	<u>0.867</u>	0.078
	GP-IRT	0.681	0.093*	0.786	0.068*	0.832	<u>0.058</u>	0.869	<u>0.052</u>	0.901	0.045*
	REPCORE	0.718	<u>0.095</u>	0.824	<u>0.069</u>	0.870	0.057	0.898	0.049	0.913	0.045
GSM8K	RANDOM	0.702	0.087	0.815	0.060	<u>0.870</u>	0.049	<u>0.898</u>	<u>0.042</u>	<u>0.917</u>	0.037
	ANCHORPOINTS	0.739	0.133	0.802	0.134	<u>0.826</u>	0.142	<u>0.831</u>	<u>0.154</u>	<u>0.840</u>	0.144
	GP-IRT	<u>0.750</u>	0.069*	<u>0.839</u>	<u>0.057</u>	0.857	0.051	0.865	<u>0.042</u>	0.896	0.034
	REPCORE	0.763	<u>0.071</u>	0.872	0.047	0.905	0.039	0.926	0.035	0.938	0.032
SEED-Bench -2-Plus	RANDOM	0.607	0.124	<u>0.728</u>	0.088	<u>0.800</u>	0.070	<u>0.836</u>	0.060	0.861	0.053
	ANCHORPOINTS	0.653	0.139	<u>0.715</u>	0.116	<u>0.747</u>	0.110	<u>0.771</u>	0.108	0.782	0.105
	GP-IRT	0.594	<u>0.098</u>	0.699	<u>0.063</u>	0.780	0.050*	0.824	0.043*	0.863	0.038*
	REPCORE	<u>0.642</u>	0.084	0.755	0.060	0.819	<u>0.051</u>	0.853	<u>0.045</u>	0.874	<u>0.040</u>

Baselines and Metrics The main experiments compare REPCORE with three primary output-based benchmark compression baselines. *Random* selects the coreset via uniform random sampling. *AnchorPoints* (Vivek et al., 2024) represents items using their binary response vectors over $\mathcal{S}$ and selects representative anchors via clustering in this discrete space. *tinyBenchmarks* (GP-IRT) (Maia Polo et al., 2024) employs a Gaussian Process-corrected Item Response Theory estimator to infer full-benchmark performance from the coreset evaluations.

To broaden the comparison with recent benchmark compression methods, we additionally report *EffiEval* (Wang et al., 2025b) and *TailoredBench* (Yuan et al., 2025) as auxiliary baselines. These two methods are evaluated under the same settings as the main experiments, while the subsequent diagnostic analyses focus on the primary baselines that are more directly tied to the controlled factors under study.

For evaluation, we report two primary metrics on the target set $\mathcal{T}$: (i) *Spearman’s rank correlation* (ρ) between the predicted model rankings and the ground-truth rankings derived from full benchmark accuracy; (ii) *Mean Absolute Error* (MAE) between the predicted and ground truth accuracies.

3.2. Main Results

We evaluate benchmark compression performance in the source-scarce regime with $|\mathcal{S}| = 10$. Table 1 summarizes the results on BBH, GSM8K, and SEED-Bench-2-Plus. The remaining two benchmarks, ARC Challenge and MMLU-Pro, show consistent trends and are reported in Appendix C.2. We further provide auxiliary comparisons with EffiEval and TailoredBench in Appendix C.3.

REPCORE: Effective Ranking and Reliable Extrapolation under Source Scarcity As shown in Table 1, REPCORE consistently achieves stronger recovery of full-benchmark rankings. By leveraging consensus embeddings to recover the geometry of the item space, our method selects representative items even at small coreset sizes. For example, on GSM8K with $K = 20$, REPCORE attains a Spearman correlation of 0.872, surpassing GP-IRT at 0.839 and ANCHORPOINTS at 0.802. In terms of absolute performance estimation, REPCORE typically yields the lowest mean absolute error and improves steadily as K increases, maintaining a clear margin over baselines. Furthermore, extended analyses in Appendix C.4 indicate that REPCORE exhibits lower variance across experimental runs, demonstrating greater stability compared to baseline methods.

Analysis of Prediction Fidelity We observe that GP-IRT occasionally attains competitive MAE scores, marked with *. However, this aggregate metric can mask item-level errors through cancellation. To assess prediction fidelity at the item level, we report the *Agreement* metric following Vivek et al. (2024), which measures the proportion of exact matches between predicted and ground-truth correctness labels. As detailed in Appendix C.1, GP-IRT exhibits substantially lower Agreement even when its MAE appears favorable. For instance, on BBH with $K = 20$, while GP-IRT matches REPCORE in MAE, its Agreement trails ours by approximately 9 percentage points. This gap indicates that REPCORE better preserves the target model’s item-level correctness pattern.

4. Analysis of Aligned Representations

While the previous sections demonstrate the effectiveness of REPCORE in benchmark compression, a critical ques-

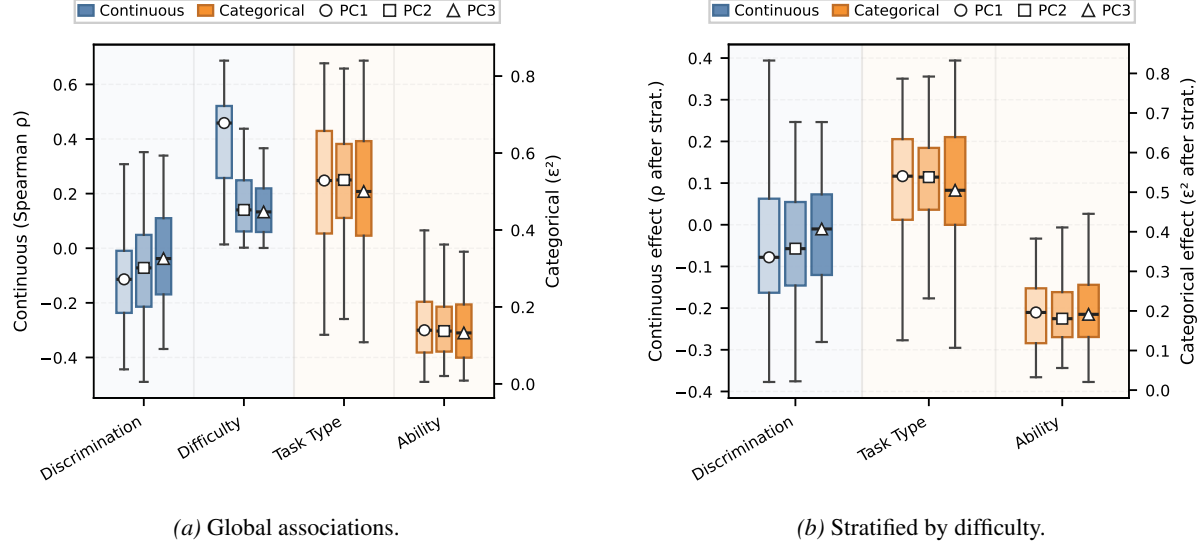


Figure 3. Factor-association analysis of the REPCORE latent space on BBH. Dual axes quantify associations: the left axis reports Spearman’s ρ for continuous factors (blue), while the right axis denotes effect size ϵ^2 for categorical factors (orange). Markers indicate principal components (Circle: PC₁, Square: PC₂, Triangle: PC₃). (a) **Global Analysis:** The primary axis (PC₁) is predominantly aligned with item difficulty ($\rho \approx 0.46$), while categorical task identities also show strong initial associations. (b) **Stratified Analysis:** After controlling for difficulty through binning, fine-grained Task Type remains strongly clustered ($\epsilon^2 \approx 0.54$), and this persistence further extends to coarse Ability Mode ($\epsilon^2 \approx 0.20$), confirming that the latent space captures task-specific structure beyond scalar difficulty.

tion remains regarding the interpretability of the learned representations: *Does the latent space merely act as a proxy for item difficulty, or does it capture richer, task-specific structural information?* To investigate this question, we analyze the geometric structure of the latent space in Section 4.2 and evaluate the utility of its principal components in Section 4.3.

4.1. Geometric Definition and Statistical Protocol

We conduct our primary analysis on benchmark **Big-Bench Hard** to leverage its hierarchical taxonomy, which divides items into fine-grained `subtasks` and broader `ability` modes (see Appendix B.3 for detailed category definitions). This structure enables us to probe the granularity of the learned item representations.

Analysis Protocol We analyze the geometric properties of aligned embeddings across 100 model snapshots (10 combos $\times$ 10 models, aligned with the main experiment). The protocol proceeds in three steps:

1. Embedding Projection and Decomposition For each model snapshot m and item i , we first compute the aligned embedding $z_{m,i} = f_\theta(\text{Proj}_m(h_{m,i})) \in \mathbb{R}^{32}$. These vectors are then aggregated over all items to form the embedding matrix $Z_m \in \mathbb{R}^{|I| \times 32}$, and we denote the empirical mean of these embeddings as $\bar{z}_m$. Next, we decompose the centered data via PCA to derive the k -th principal axis $w_m^{(k)}$. Finally, the projection score $\xi_{m,i}^{(k)}$ is computed by projecting the centered embedding onto the k -th principal

axis: $\xi_{m,i}^{(k)} = \langle z_{m,i} - \bar{z}_m, w_m^{(k)} \rangle$. Averaged over the analyzed BBH snapshots, the top three principal components explain 50.12%, 16.21%, and 9.21% of the variance, respectively.

2. Interpretable Factors To investigate whether the learned representation recovers the intrinsic structural relationships among items in the benchmark, we evaluate the alignment between the projection scores $\xi_{m,i}^{(k)}$ and four ground-truth attributes. Detailed computational formulations and definitions for these attributes are provided in Appendix B.3 and B.4:

- **Continuous Factors:** Difficulty δ_i (pass rate) and Discrimination γ_i (Kelley 27% D).
- **Discrete Factors:** Subtask Identity τ_i (fine-grained) and Ability Mode α_i (coarse-grained).

3. Statistical Association To rigorously quantify the alignment between distinct principal components and the item attributes defined above, we propose a dual-granularity evaluation protocol centered on a unified association operator $\mathcal{M}(\cdot)$. This operator is realized as Spearman’s ρ for continuous attributes (δ, γ) and Kruskal-Wallis ϵ^2 for categorical ones (τ, α), thereby providing a consistent measure to evaluate the representational geometry across two distinct granularities:

- **Global Analysis:** For a tested item attribute $\psi \in \{\delta, \gamma, \tau, \alpha\}$, let ψ_i denote its value for item i . The macro-scale association is evaluated over the entire benchmark

$\mathcal{I}$ to capture general trends:

$$\mathcal{A}_{\text{global}}^{(k,m)} = \mathcal{M}\left(\left\{\xi_{m,i}^{(k)}\right\}_{i \in \mathcal{I}}, \left\{\psi_i\right\}_{i \in \mathcal{I}}\right) \quad (9)$$

- **Stratified Analysis:** To isolate structural signals from the confounding effects of item difficulty, we partition $\mathcal{I}$ into $B = 20$ equi-sized bins $\{\mathcal{I}_b\}_{b=1}^B$ based on δ . Local statistics are then aggregated for the remaining attributes to derive a difficulty-invariant estimate:

$$\mathcal{A}_{\text{strat}}^{(k,m)} = \text{Agg}\left(\left\{\mathcal{M}\left(\left\{\xi_{m,i}^{(k)}\right\}_{i \in \mathcal{I}_b}, \left\{\psi_i\right\}_{i \in \mathcal{I}_b}\right)\right\}_{b=1}^B\right) \quad (10)$$

where $\text{Agg}(\cdot)$ aggregates p -values using Stouffer’s method and effect sizes using statistic-specific procedures, including Fisher-z transformation for Spearman correlations and sample-size-weighted averaging for Kruskal-Wallis ϵ^2 . Full computational details are deferred to Appendix B.1.

To mitigate sampling variance, we aggregate these metrics over the full snapshot pool. Figure 3 summarizes the resulting distributions via medians and interquartile ranges, offering a robust characterization of the representational geometry. Quantitatively, this characterization is statistically rigorous: both stratified p -values and FDR-corrected q -values consistently satisfy significance thresholds ($\alpha = 0.05$) across snapshots, with medians often approaching floating-point limits (see Appendix B.2 for full results).

4.2. Disentangling Difficulty and Structure

Global View: The Dominance of Difficulty Globally (Figure 3a), PC_1 emerges as the dominant axis of variation, manifesting a strong correlation with item difficulty ($\rho \approx 0.46$). While categorical factors such as `subtask type` exhibit substantial effect sizes ($\epsilon^2 \approx 0.53$), we observe that this alignment is potentially susceptible to difficulty confounding. Specifically, if items within the same subtask share homogeneous complexity levels, global clustering may be a misleading artifact of difficulty rather than a reflection of task-specific reasoning patterns. This possibility necessitates a more granular investigation to adjudicate whether the captured signals are truly structural.

Stratified View: Hierarchical Structural Persistence To verify the persistence of structural identity, we move to a difficulty-stratified analysis (Figure 3b), which serves as a controlled setting to isolate task-specific signals from the aforementioned confounding effects. The results indicate that even within narrow difficulty bins, clustering by `subtask type` remains remarkably robust ($\epsilon^2 \approx 0.54$). This resilience confirms that the aligned representations $z_{m,i}$ encode fine-grained functional identities that transcend mere performance-based scalar signals. Moreover, this persistence further scales to the benchmark’s hierarchy, with the `ability mode` alignment ($\epsilon^2 \approx 0.20$) confirming the recovery of abstract reasoning domains. These results sug-

gest that the latent manifold preserves high-level cognitive modalities beyond granular task labels.

The Role of Discrimination In contrast to the dominant structural geometry, item discrimination acts as a secondary background signal, maintaining a modest yet stable negative correlation with PC_1 across both global ($\rho \approx -0.11$) and stratified ($\rho \approx -0.08$) regimes. This suggests that while the bottleneck primarily optimizes for difficulty and structural features, it retains auxiliary information regarding the reliability of the item representations.

Table 2. Principal component ablation on BBH for performance extrapolation across $K \in \{10, 20, 30, 40, 50\}$.

$\rho \uparrow$	$K = 10$	$K = 20$	$K = 30$	$K = 40$	$K = 50$
PC_1	0.665	0.793	0.857	0.888	0.907
PC_2	0.662	0.794	0.853	0.887	0.907
PC_3	0.689	0.812	0.859	0.882	0.900
$\text{PC}_{1,2}$	<u>0.689</u>	<u>0.812</u>	<u>0.871</u>	0.898	0.917
$\text{PC}_{1,2,3}$	0.686	<u>0.820</u>	0.875	<u>0.896</u>	<u>0.914</u>
ALL	0.718	0.824	0.870	0.898	0.913
MAE $\downarrow$	$K = 10$	$K = 20$	$K = 30$	$K = 40$	$K = 50$
PC_1	<u>0.101</u>	0.074	0.060	0.053	0.048
PC_2	<u>0.101</u>	<u>0.070</u>	<u>0.058</u>	<u>0.050</u>	0.045
PC_3	0.103	0.072	0.060	0.052	0.047
$\text{PC}_{1,2}$	0.104	0.072	0.059	0.051	0.045
$\text{PC}_{1,2,3}$	0.104	0.073	<u>0.058</u>	0.051	<u>0.046</u>
ALL	0.095	0.069	0.057	0.049	0.045

4.3. Utility of the Interpretable Subspace

Building on the structural insights from the spectral analysis, we further evaluate how these interpretable dimensions contribute to the core task of performance extrapolation. Table 2 presents the ablation results on the BBH benchmark, where we compare the predictive efficacy of individual principal components and low-rank subspaces against the full aligned representations across varying coreset budgets.

Beyond Scalar Difficulty While PC_1 provides a robust predictive baseline by capturing item difficulty, the inclusion of PC_2 and PC_3 consistently enhances estimation fidelity. This improvement reinforces our geometric findings: higher-order components capture essential structural context including task identity and reasoning modalities. By leveraging these signals, the regressor can adjust for model-specific capabilities that a standard difficulty curve cannot account for.

Robustness and Signal Concentration A comparison with the full aligned embedding space reveals a distinct performance crossover. In extremely data-scarce regimes where $K = 10$, the full aligned embedding slightly outperforms the $\text{PC}_{1,2}$ subspace. However, as the coreset size grows to $K \geq 30$, the $\text{PC}_{1,2}$ subspace demonstrates superior

robustness and eventually surpasses the full aligned embedding space. This shift suggests that while high-dimensional features may offer some utility at minimal sample sizes, they also introduce predictive instability. Conversely, the low-rank subspace concentrates the most generalizable signals to provide a stable basis for extrapolation without relying on the noise inherent in the high-dimensional manifold.

5. Ablations and Additional Analyses

Continuous Representations Enable Superior Coreset Selection To validate the necessity of our representation learning module, we conduct a controlled experiment by substituting the consensus embeddings with binary correctness vectors during the clustering phase, while maintaining the identical downstream extrapolation mechanism. Table 4 reports the average performance with $K = 30$. The results indicate that relying on discrete correctness signals yields inferior ranking correlation and higher estimation error. This performance gap confirms that the continuous embedding space preserves fine-grained item distinctions essential for representative sampling, which are not captured by binary correctness signals. Detailed per-dataset results are provided in Appendix C.6.

Table 4. Ablation on clustering inputs averaged over five benchmarks with $K = 30$.

Clustering Input	$\rho \uparrow$	MAE $\downarrow$
Correctness Vectors	0.850	0.055
Consensus Embeddings	0.868	0.049

Aligned Representations Are Data-Efficient in Source-Scarce Regimes We evaluate the data efficiency of our framework by varying the source pool size $|\mathcal{S}|$. Table 3 presents the performance metrics as the number of source models increases from 5 to 20 with $K = 30$. REPCORE demonstrates strong stability even in highly resource-constrained scenarios. Under the most restricted setting with only five source models, output-based methods, including ANCHORPOINTS, exhibit a substantial performance drop. In contrast, REPCORE achieves a Spearman correlation of 0.860, which significantly outperforms baselines. Furthermore, the performance of our framework stabilizes rapidly as the source pool expands. This saturation indicates that the aligned space effectively recovers the geometry of the item space under minimal supervision, thereby reducing the dependency on extensive source model pools. Detailed per-dataset results are provided in Appendix C.9.

Impact of Source Model Composition We investigate how source pool diversity affects compression quality. Table 5 indicates that a diverse composition of model families and capability ranges yields the most robust performance.

Relying on a single model family introduces architectural biases, as evidenced by the performance drop in single-family pools. Similarly, homogeneous pools restricted to a narrow capability range prove suboptimal, as they fail to simultaneously maintain high ranking correlation and low estimation error. These findings suggest that incorporating diverse sources mitigates architectural and capability biases, allowing the aligned representations to better capture the geometry of the item space that supports reliable extrapolation. Detailed per-dataset results are provided in Appendix C.10 and Appendix C.11.

Table 5. Performance comparison under varied source compositions averaged across five benchmarks at $K = 30$.

Composition	Setting	$\rho \uparrow$	MAE $\downarrow$
<i>Model Family</i>	Family A [†]	0.848	0.057
	Family B [‡]	0.686	0.058
	Diverse (Ours)	0.868	0.049
<i>Capability</i>	Strong Only	0.778	0.092
	Weak Only	0.799	0.055
	Diverse (Ours)	0.868	0.049

[†] Qwen / Ovis; [‡] Llama / InternVL.

Scalability to Larger Coreset Budgets We further examine scalability by increasing the coreset budget to $K = 100$, 150, and 200. As shown in Table 6, REPCORE improves steadily as K increases and maintains strong ranking and estimation accuracy relative to baselines. These results indicate that the learned item space continues to support effective coreset selection at larger budgets. Detailed per-dataset results are provided in Appendix C.12.

Table 6. Average performance across benchmarks with larger coreset budgets.

Method	$K = 100$		$K = 150$		$K = 200$	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
RANDOM	0.947	0.033	0.964	0.027	0.972	0.023
ANCHORPOINTS	0.833	0.111	0.832	0.107	0.836	0.099
GP-IRT	0.951	0.027	0.966	0.023	0.973	0.021
REPCORE	0.954	0.028	0.967	0.023	0.975	0.021

Robustness to Temporal Distribution Shift Rapid progress in LLMs induces temporal shifts in model behaviors. To simulate leaderboard-style evaluation, we sort models by their release time and use the oldest 30 models as the historical source candidate pool. For each benchmark, we conduct 10 trials, each sampling 10 source models from this pool and holding out all remaining models as newer targets. As shown in Table 7, REPCORE achieves the strongest ranking performance with competitive estimation error, suggesting that the learned item space remains effective as the

Table 3. Performance averaged over five benchmarks for varying source pool sizes with $K = 30$.

Method	$ \mathcal{S} = 5$		$ \mathcal{S} = 10$		$ \mathcal{S} = 15$		$ \mathcal{S} = 20$	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
ANCHORPOINTS	0.640	0.210	0.809	0.116	0.848	0.091	0.861	0.080
GP-IRT	0.822	0.057	0.836	0.053	0.843	0.049	0.828	0.050
REPCORE	0.860	0.052	0.868	0.049	0.860	0.049	0.864	0.049

model pool evolves. Detailed per-dataset results are provided in Appendix C.13.

 Table 7. Temporal generalization under a chronological split at $K = 30$, averaged over 10 trials across five benchmarks.

Method	$\rho \uparrow$	MAE $\downarrow$
RANDOM	0.845	0.060
ANCHORPOINTS	0.797	0.110
GP-IRT	0.835	0.049
REPCORE	0.853	0.050

6. Discussion of Method Boundaries

6.1. Dependence on Hidden States

Table 8. Masking results under partial source hidden-state unavailability. Results are averaged over five benchmarks and five coreset budgets, with output-based baselines included for reference.

Masked Sources	Random Mask		Advdist Mask	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
0%	0.842	0.054	—	—
10%	0.842	0.058	0.840	0.058
20%	0.839	0.058	0.836	0.060
30%	0.837	0.058	0.828	0.061
40%	0.837	0.059	0.820	0.062
50%	0.833	0.059	0.805	0.065
Random Subset	0.818	0.070	—	—
AnchorPoints	0.791	0.120	—	—
tinyBenchmarks (GP-IRT)	0.811	0.057	—	—
TailoredBench	0.747	0.075	—	—
EffiEval	0.762	0.111	—	—

REPCORE assumes access to hidden states of the source models used for coreset construction. This restricts the direct use of closed-source or API-only models as representation sources, although their output responses can still be used. To further probe this boundary, we simulate partially accessible source pools by masking hidden states for a subset of sources while retaining their benchmark responses. For masked sources, we approximate item representations from accessible sources with similar output behavior and use the resulting pseudo-representations for coreset selection.

Table 8 reports two masking methods. Random Mask removes source hidden states uniformly at random, while Advdist Mask provides a harder stress test by first masking

sources that are most distinct from the rest according to their output correctness vectors. The pseudo-representation construction and the distinctness score used by Advdist Mask are detailed in Appendix C.14.

Under random masking, performance degrades mildly and remains above output-based baselines in ranking correlation even when 50% of hidden states are unavailable. The Advdist setting reveals a clearer boundary: the advantage narrows when the inaccessible sources provide distinctive behavioral coverage. Overall, REPCORE is robust to random or redundant missingness, but benefits most when hidden states are available for behaviorally diverse sources.

6.2. Beyond Binary-Scored Benchmarks

In this paper, we primarily focus on binary-scored benchmarks, where each item is parsed into a correctness label and the alignment objective uses cross-entropy loss. This scope matches common free-generation QA evaluation protocols, but excludes settings with continuous item-level outcomes, such as graded scores or reward-model judgments. This is an empirical scope limitation rather than a fundamental restriction of REPCORE. Since coreset selection relies on hidden state geometry, it does not inherently require binary labels. For continuous metrics, Eq. (2) can be replaced with a regression loss such as MSE, and the target-specific extrapolator can be trained to predict continuous item-level scores instead of binary correctness.

7. Conclusion and Future Work

In this work, we introduce REPCORE, a framework leveraging aligned representations to identify representative benchmark subsets. Our evaluation confirms that REPCORE significantly outperforms output-based baselines in ranking correlation and estimation accuracy, while maintaining robust performance under source model scarcity and temporal distribution shifts. For future directions, we aim to extend this paradigm to closed-source models, thereby establishing a unified standard for open and proprietary systems. Furthermore, we aim to extend REPCORE to more general benchmark settings beyond binary-scored evaluation and explore the cross-domain transferability of aligned representations, enabling efficient coreset selection for broader evaluation protocols and specialized domains.

Acknowledgements

This work is supported by the Beijing Natural Science Foundation (Grant Nos. 4262065, 4222037, and L181010).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. [arXiv preprint arXiv:2303.08774](#), 2023.
- Azaria, A. and Mitchell, T. The internal state of an llm knows when it’s lying. In [Findings of the Association for Computational Linguistics: EMNLP 2023](#), pp. 967–976, 2023.
- Burns, C., Ye, H., Klein, D., and Steinhardt, J. Discovering latent knowledge in language models without supervision. In [International Conference on Learning Representations \(ICLR\)](#), 2023.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. [arXiv preprint arXiv:1803.05457](#), 2018.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. [arXiv preprint arXiv:2110.14168](#), 2021.
- Gekhman, Z., Ben-David, E., Orgad, H., Ofek, E., Belinkov, Y., Szpektor, I., Herzig, J., and Reichart, R. Inside-out: Hidden factual knowledge in LLMs. In [Second Conference on Language Modeling](#), 2025. URL <https://openreview.net/forum?id=f7GG1MbsSM>.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. [arXiv preprint arXiv:2407.21783](#), 2024.
- Gupta, V., Ross, C., Pantoja, D., Passonneau, R. J., Ung, M., and Williams, A. Improving model evaluation using smart filtering of benchmark datasets. In [Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies \(Volume 1: Long Papers\)](#), pp. 4595–4615, 2025.
- Li, B., Ge, Y., Chen, Y., Ge, Y., Zhang, R., and Shan, Y. Seed-bench-2-plus: Benchmarking multimodal large language models with text-rich visual comprehension. [arXiv preprint arXiv:2404.16790](#), 2024.
- Li, K., Patel, O., Viégas, F., Pfister, H., and Wattenberg, M. Inference-time intervention: Eliciting truthful answers from a language model. [Advances in Neural Information Processing Systems](#), 36:41451–41530, 2023.
- Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Re, C., Acosta-Navas, D., Hudson, D. A., Zelikman, E., Durmus, E., Ladhak, F., Rong, F., Ren, H., Yao, H., WANG, J., Santhanam, K., Orr, L., Zheng, L., Yuksekgonul, M., Suzgun, M., Kim, N., Guha, N., Chatterji, N. S., Khat-tab, O., Henderson, P., Huang, Q., Chi, R. A., Xie, S. M., Santurkar, S., Ganguli, S., Hashimoto, T., Icard, T., Zhang, T., Chaudhary, V., Wang, W., Li, X., Mai, Y., Zhang, Y., and Koreeda, Y. Holistic evaluation of language models. [Transactions on Machine Learning Research](#), 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=iO4LZibEqW>. Featured Certification, Expert Certification, Outstanding Certification.
- Lugoloobi, W. and Russell, C. Llms encode how difficult problems are. [arXiv preprint arXiv:2510.18147](#), 2025.
- Ma, Z., Yuan, Q., Wang, Z., and Zhou, D. Large language models have intrinsic meta-cognition, but need a good lens. In [Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing](#), pp. 3460–3477, 2025.
- Maia Polo, F., Weber, L., Choshen, L., Sun, Y., Xu, G., and Yurochkin, M. tinyBenchmarks: evaluating LLMs with fewer examples. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), [Proceedings of the 41st International Conference on Machine Learning](#), volume 235 of [Proceedings of Machine Learning Research](#), pp. 34303–34326. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/maia-polo24a.html>.
- Marjanović, S. V., Patel, A., Adlakha, V., Aghajohari, M., BehnamGhader, P., Bhatia, M., Khandelwal, A., Kraft, A., Krojer, B., Lù, X. H., et al. DeepSeek-R1 thoughtology: Let’s think about llm reasoning. [arXiv preprint arXiv:2504.07128](#), 2025.
- Orgad, H., Toker, M., Gekhman, Z., Reichart, R., Szpektor, I., Kotek, H., and Belinkov, Y. LLMs know more than they show: On the intrinsic representation of LLM hallucinations. In [The Thirteenth International Conference](#)

- on Learning Representations, 2025. URL <https://openreview.net/forum?id=KRnsX5Em3W>.
- Perlit, Y., Bandel, E., Gera, A., Arviv, O., Ein-Dor, L., Shnarch, E., Slonim, N., Shmueli-Scheuer, M., and Choshen, L. Efficient benchmarking (of language models). In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 2519–2536, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.139. URL <https://aclanthology.org/2024.naacl-long.139/>.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q., Chi, E., Zhou, D., et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Vivek, R., Ethayarajh, K., Yang, D., and Kiela, D. Anchor points: Benchmarking models with much fewer examples. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1576–1601, 2024.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.
- Wang, Y., Li, H., Zou, H., Zhang, J., He, X., Li, Q., and Xu, K. FacLens: Transferable probe for foreseeing non-factuality in fact-seeking question answering of large language models. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 18563–18582, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.937. URL <https://aclanthology.org/2025.emnlp-main.937/>.
- Wang, Y., Ying, J., Cao, Y., Ma, Y., and Jiang, Y. Effieval: Efficient and generalizable model evaluation via capability coverage maximization. *arXiv preprint arXiv:2508.09662*, 2025b.
- Wang, Y., Zhang, P., Yang, B., Wong, D. F., and Wang, R. Latent space chain-of-embedding enables output-free LLM self-evaluation. In *The Thirteenth International Conference on Learning Representations*, 2025c. URL <https://openreview.net/forum?id=jxo70B9fQo>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yuan, P., Zhang, Y., Feng, S., Li, Y., Wang, X., Shi, J., Tan, C., Pan, B., Hu, Y., and Li, K. Beyond one-size-fits-all: Tailored benchmarks for efficient evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15591–15615, 2025.
- Zhang, A., Chen, Y., Pan, J., Zhao, C., Panda, A., Li, J., and He, H. Reasoning models know when they’re right: Probing hidden states for self-verification. *arXiv preprint arXiv:2504.05419*, 2025.
- Zhu, Y., Liu, D., Lin, Z., Tong, W., Zhong, S., and Shao, J. The llm already knows: Estimating llm-perceived question difficulty via hidden representations. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 1160–1176, 2025.
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A. Related Work

A.1. Benchmark Compression

Benchmark Compression aims to select a minimal subset of items that preserves full-benchmark rankings and enables accurate score estimation. Existing methods can be categorized based on the signals they utilize: prediction outputs versus input/internal features.

Prediction-Based Coreset Selection. The dominant paradigm relies on the correlation of model outputs (i.e., accuracy) across test items. ANCHORPOINTS (Vivek et al., 2024) and TAILOREDBENCH (Yuan et al., 2025) represent each item by its “prediction pattern”—a vector of correctness scores derived from a population of source models. By applying clustering to these patterns, they identify items that maximize information gain. Similarly, TINYBENCHMARKS (Maia Polo et al., 2024) adopts Item Response Theory (IRT) to fit latent discrimination and difficulty parameters from response patterns. A critical limitation of these methods is the “cold start” problem: they require a diverse pool of source models to construct robust prediction patterns. In low-resource regimes (e.g., few source models), the sparse binary signal (correct/incorrect) is often insufficient to capture fine-grained dataset structures.

Feature-Based and Heuristic Filtering. Alternative approaches utilize static semantic features or internal model statistics. SMART (Gupta et al., 2025) filters datasets by removing samples that are semantically redundant (measured via textual embedding distance) or statistically “easy” for models. More recently, EFFIEVAL (Wang et al., 2025b) proposes a training-free selection method that maximizes the coverage of activated neurons, positing that diverse activation patterns correspond to diverse model capabilities. While EFFIEVAL leverages internal signals, it focuses on neuron-level coverage for sampling rather than aligning latent spaces for structural recovery. Our work advances this direction by exploiting the continuous hidden states to characterize item structure, providing a denser signal than discrete outputs while avoiding the need for massive source model populations.

A.2. Internal Representations and Difficulty Perception

Beyond their use in compression, hidden states have been extensively studied for their ability to encode task-level properties.

Probing for Factuality. Representation engineering has shown that LLM hidden states contain linearly separable directions corresponding to truthfulness. Methods like SAPLMA (Azaria & Mitchell, 2023) and FACLENS (Wang et al., 2025a) train probes to detect hallucinations or predict correctness before generation is complete, suggesting models possess intrinsic meta-cognition regarding their knowledge boundaries (Ma et al., 2025; Orgad et al., 2025).

Encoding Difficulty and Structure. Recent findings indicate that hidden representations also implicitly encode the complexity of input queries. Zhu et al. (2025) and Lugoloobi & Russell (2025) demonstrate that the difficulty of a question can be estimated directly from its hidden states or attention patterns, often providing a more reliable signal of model-specific difficulty than the model’s own explicit judgments. Unlike prior works that use these signals primarily for difficulty estimation or inference-time intervention (Li et al., 2023), we propose REPCORE to leverage these high-dimensional, density-rich signals for benchmark compression. By treating hidden states as a continuous proxy for item characteristics, we can recover dataset structures more effectively than methods relying on sparse discrete labels.

B. Methodological Details

B.1. Aggregation Operator for Stratified Associations

This section formalizes the aggregation operator $\text{Agg}(\cdot)$ used in the stratified analysis protocol (Section 4.1). Let $\{\mathcal{I}_b\}_{b=1}^B$ denote the difficulty-stratified partition of the benchmark item set $\mathcal{I}$. For each bin $\mathcal{I}_b$, the within-bin procedure $\mathcal{M}(\cdot)$ produces a pair (η_b, p_b) , where η_b is an effect-size statistic and p_b is the corresponding p -value (bins with undefined outputs are excluded from aggregation). Let n_b be the number of valid items contributing to $\mathcal{M}(\cdot)$ in bin b . To account for varying bin sample sizes in the aggregation, we assign each bin the weight $w_b = \sqrt{n_b}$.

Aggregation of p -values (weighted Stouffer) Let p_b denote the p -value computed on bin b . We convert p_b to a normal score s_b by

$$s_b = \begin{cases} \Phi^{-1}(1 - p_b), & \text{Kruskal–Wallis (one-sided),} \\ \Phi^{-1}\left(1 - \frac{p_b}{2}\right), & \text{Spearman (two-sided).} \end{cases} \quad (11)$$

The weighted Stouffer statistic is

$$Z_{\text{st}} = \frac{\sum_b w_b s_b}{\sqrt{\sum_b w_b^2}}. \quad (12)$$

Accordingly, the stratified p -value is

$$p_{\text{strat}} = \begin{cases} 1 - \Phi(Z_{\text{st}}), & \text{Kruskal–Wallis,} \\ 2(1 - \Phi(|Z_{\text{st}}|)), & \text{Spearman,} \end{cases} \quad (13)$$

where $\Phi(\cdot)$ denotes the standard normal CDF.

Aggregation of effect sizes Effect sizes are aggregated separately for continuous and discrete attributes.

- **Continuous attributes (Spearman).** Let ρ_b denote the Spearman correlation computed on bin b . Correlations are combined via the Fisher transformation with inverse-variance weights:

$$\zeta = \frac{\sum_b (n_b - 3) \operatorname{arctanh}(\rho_b)}{\sum_b (n_b - 3)}, \quad (14)$$

$$\rho_c = \tanh(\zeta), \quad (15)$$

where $(n_b - 3)$ is the standard inverse-variance weight under Fisher’s transformation. The aggregated correlation ρ_c is reported as the stratified effect size.

- **Discrete attributes (Kruskal–Wallis).** Let H_b denote the Kruskal–Wallis statistic computed on bin b , using n_b valid items across g_b retained groups. The within-bin effect size is quantified by epsilon-squared:

$$\epsilon_b^2 = \max\left\{0, \frac{H_b - g_b + 1}{n_b - g_b}\right\}. \quad (16)$$

The stratified effect size is the mean of within-bin estimates, weighted by their respective sample sizes n_b :

$$\epsilon_{\text{strat}}^2 = \frac{\sum_b n_b \epsilon_b^2}{\sum_b n_b}. \quad (17)$$

Multiple testing control Let Ψ denote the set of tested item attributes. For each fixed principal component index k , we apply Benjamini–Hochberg FDR correction to the collection of stratified p -values $\{p_{\text{strat}}(\psi)\}_{\psi \in \Psi}$. The resulting per-component adjusted values are reported as $q_{\text{strat}, \text{pcwise}}$.

B.2. Detailed Statistical Distributions of Aligned Representations

To complement the summary statistics provided in the main text, we report the detailed distribution of attribute-aggregated significance metrics across the 100 model snapshots. For each principal component, global significance statistics are aggregated over all tested item attributes $\psi \in \{\delta, \gamma, \tau, \alpha\}$. For stratified statistics, difficulty δ is used only to construct the bins and is therefore excluded from the attribute aggregation; the reported stratified metrics are aggregated over $\psi \in \{\gamma, \tau, \alpha\}$. Table 9 presents the median, 25th percentile, and 75th percentile of the raw global p -values and the stratified significance metrics, including both raw p -values and FDR-corrected q -values (q_{strat}).

As observed, the associations are overwhelmingly significant. For the difficulty-stratified analysis, the median q_{strat} values for the first three principal components (PC₁–PC₃) are consistently below 10^{-190} , approaching the numerical precision limits of standard floating-point representations. These results confirm that the geometric alignment captured by REPCORE is statistically robust and not an artifact of specific source model sampling.

Table 9. Distribution of attribute-aggregated statistical significance metrics. We report the empirical distribution (median, 25th, and 75th percentiles) of significance tests across 100 model snapshots. Global metrics are aggregated over all tested item attributes $\psi \in \{\delta, \gamma, \tau, \alpha\}$ and reported as raw p -values from the corresponding association tests. For stratified metrics, difficulty δ is used only for binning, the aggregation is performed over $\psi \in \{\gamma, \tau, \alpha\}$, and both raw p -values and FDR-corrected q -values (q_{strat}) are reported. Note that values displayed as 0 indicate probabilities below the numerical precision limit (underflow).

Mode	Component	Metric	Median	25% Percentile	75% Percentile	IQR
Global	PC1	p	4.16×10^{-249}	0	6.96×10^{-70}	6.96×10^{-70}
	PC2	p	2.07×10^{-129}	0	2.02×10^{-30}	2.02×10^{-30}
	PC3	p	7.46×10^{-109}	0	3.96×10^{-26}	3.96×10^{-26}
Stratified	PC1	p	1.27×10^{-204}	0	3.41×10^{-65}	3.41×10^{-65}
		q_{strat}	3.18×10^{-204}	0	5.69×10^{-65}	5.69×10^{-65}
	PC2	p	1.76×10^{-193}	0	4.56×10^{-60}	4.56×10^{-60}
		q_{strat}	4.39×10^{-193}	0	5.88×10^{-60}	5.88×10^{-60}
	PC3	p	5.25×10^{-203}	0	1.57×10^{-59}	1.57×10^{-59}
		q_{strat}	1.31×10^{-202}	0	1.96×10^{-59}	1.96×10^{-59}

B.3. BBH Taxonomy and Ability Mapping

Table 10 summarizes the BBH subtask grouping used in our analysis. Since BBH contains heterogeneous subtasks with different reasoning demands, we consolidate the fine-grained task names into five broader ability modes (Logical, Language, Mathematical, Spatial/Temporal, and Knowledge/Domain). This mapping provides a consistent level of abstraction for reporting and interpreting stratified results at the ability level, while retaining the original BBH taxonomy at the subtask level.

Table 10. BBH taxonomy and task grouping.

Ability	Subtasks
Logical	boolean_expressions, dyck_languages, formal_fallacies, web_of_lies, logical_deduction_{three, five, seven}_objects
Language	disambiguation_qa, hyperbaton, ruin_names, salient_translation_error_detection, snarks
Mathematical	geometric_shapes, multistep_arithmetic_two, object_counting, word_sorting
Spatial/Temporal	date_understanding, navigate, temporal_sequences, tracking_shuffled_objects_{three, five, seven}_objects
Knowledge/Domain	causal_judgement, movie_recommendation, penguins_in_a_table, reasoning_about_colored_objects, sports_understanding

B.4. Computation of Continuous Factors

All continuous item factors are computed from responses of the full evaluated model set $\mathcal{U} = \mathcal{S} \cup \mathcal{T}$. Let $a_{m,i} \in \{0, 1\}$ indicate whether model $m \in \mathcal{U}$ answers item i correctly.

Difficulty (δ_i). Item difficulty is defined as the pass rate across the full evaluated models:

$$\delta_i = \frac{1}{|\mathcal{U}|} \sum_{m \in \mathcal{U}} a_{m,i}. \quad (18)$$

Lower values of δ_i correspond to more difficult items.

Discrimination (γ_i ; Kelley 27% D) Item discrimination is defined using Kelley’s 27% high–low criterion. Each model is first assigned an overall benchmark score

$$r_m = \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} a_{m,j}, \quad (19)$$

where $\mathcal{I}$ denotes the item set of the benchmark. Let $q = \max\{1, \lfloor 0.27|\mathcal{U}| \rfloor\}$, and let $\mathcal{U}_{\text{high}}$ and $\mathcal{U}_{\text{low}}$ denote the top- q and bottom- q models under r_m , respectively. For each item i , the high/low-group pass rates and the Kelley discrimination score are defined as

$$\begin{aligned} \pi_i^{\text{high}} &= \frac{1}{|\mathcal{U}_{\text{high}}|} \sum_{m \in \mathcal{U}_{\text{high}}} a_{m,i}, \\ \pi_i^{\text{low}} &= \frac{1}{|\mathcal{U}_{\text{low}}|} \sum_{m \in \mathcal{U}_{\text{low}}} a_{m,i}, \\ \gamma_i &= D_i = \pi_i^{\text{high}} - \pi_i^{\text{low}}. \end{aligned} \quad (20)$$

A higher γ_i signifies a more pronounced separation between models with high and low performance.

C. More Experimental Results

C.1. Detailed Item-level Agreement Results

Motivated by an observation in the main results that GP-IRT can achieve deceptively low MAE, Table 11 reports an **item-level Agreement** metric across budgets $K \in \{10, 20, 30, 40, 50\}$, measuring the fraction of items whose predicted correctness labels exactly match the ground truth. Agreement reveals that a low MAE does not necessarily imply faithful extrapolation: MAE can be artificially reduced when a method shrinks predictions toward the global mean or benefits from error cancellation.

Across all benchmarks and budgets, REPCORE consistently achieves the best Agreement, indicating that our predictions preserve the fine-grained per-item error pattern rather than merely matching averages. Therefore, we treat Agreement as a complementary fidelity check and caution against using MAE alone as the primary indicator of extrapolation quality.

Table 11. Item-level Agreement across budgets. Agreement ($\uparrow$) over $K \in \{10, 20, 30, 40, 50\}$; RANDOM is excluded as it does not extrapolate beyond the coreset. **Bold** and underline denote the best and second-best results.

Benchmark	Method	$K = 10$	$K = 20$	$K = 30$	$K = 40$	$K = 50$
ARC-Challenge	ANCHORPOINTS	0.760	0.741	0.736	0.730	0.731
	GP-IRT	<u>0.811</u>	<u>0.806</u>	<u>0.806</u>	<u>0.822</u>	<u>0.838</u>
	REPCORE	0.848	0.858	0.862	0.865	0.867
BBH	ANCHORPOINTS	0.612	0.626	0.630	0.634	0.634
	GP-IRT	<u>0.618</u>	<u>0.631</u>	<u>0.645</u>	<u>0.672</u>	<u>0.693</u>
	REPCORE	0.697	0.714	0.721	0.725	0.727
GSM8K	ANCHORPOINTS	0.767	0.756	0.747	0.733	0.745
	GP-IRT	<u>0.820</u>	<u>0.824</u>	<u>0.835</u>	<u>0.851</u>	<u>0.863</u>
	REPCORE	0.863	0.876	0.881	0.884	0.886
MMLU-Pro	ANCHORPOINTS	0.603	0.627	0.629	0.629	0.631
	GP-IRT	<u>0.611</u>	<u>0.651</u>	<u>0.685</u>	<u>0.713</u>	<u>0.730</u>
	REPCORE	0.726	0.744	0.751	0.754	0.756
SEED-Bench -2-Plus	ANCHORPOINTS	<u>0.654</u>	<u>0.667</u>	<u>0.670</u>	0.669	0.671
	GP-IRT	0.594	0.618	<u>0.670</u>	<u>0.718</u>	<u>0.749</u>
	REPCORE	0.769	0.784	0.791	0.794	0.797

C.2. Additional Main Results on ARC-Challenge and MMLU-Pro

Table 12 extends the main-budget evaluation to ARC-Challenge and MMLU-Pro under the same source-scarce protocol, reporting Spearman’s ρ and MAE across budgets $K \in \{10, 20, 30, 40, 50\}$. The overall trend matches the main text:

REPCORE improves monotonically with K and achieves the strongest ranking recovery once the budget leaves the extremely small regime, delivering the best ρ on both benchmarks for $K \geq 20$.

In terms of MAE, REPCORE remains competitive across budgets and often excels at small-to-moderate K , while GP-IRT can appear slightly better on MMLU-Pro at larger K . However, as discussed in Appendix C.1, MAE can be reduced by averaging effects and error cancellation. Together with Table 11, these results suggest that REPCORE achieves reliable ranking recovery while better preserving per-item correctness.

Table 12. Additional main results. We report Spearman’s rank correlation (ρ) and Mean Absolute Error (MAE). **Bold** and underline indicate the best and second-best results among competitive methods, respectively.

Benchmark	Method	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$	
		$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
ARC-Challenge	RANDOM	0.681	0.089	0.801	0.061	<u>0.856</u>	<u>0.051</u>	<u>0.888</u>	0.043	<u>0.908</u>	0.038
	ANCHORPOINTS	0.724	0.132	0.767	0.136	0.792	0.138	0.797	0.145	0.796	0.143
	GP-IRT	0.706	<u>0.076</u>	<u>0.808</u>	<u>0.057</u>	0.845	0.052	0.866	<u>0.043</u>	0.884	<u>0.036</u>
	REPCORE	<u>0.707</u>	0.072	0.820	0.050	0.871	0.041	0.900	0.036	0.921	0.032
MMLU-Pro	RANDOM	0.702	0.124	0.812	0.088	0.865	0.071	0.895	0.062	0.914	0.055
	ANCHORPOINTS	0.754	0.140	0.800	0.107	0.846	0.102	0.861	0.095	0.867	0.095
	GP-IRT	0.646	<u>0.101</u>	<u>0.806</u>	0.065*	<u>0.866</u>	0.053*	<u>0.899</u>	0.046*	<u>0.920</u>	0.041*
	REPCORE	<u>0.732</u>	0.091	0.829	<u>0.067</u>	0.875	<u>0.055</u>	0.906	<u>0.048</u>	0.924	<u>0.043</u>

C.3. Additional Baselines Results

Table 13. Mean performance: REPCORE (ours) vs. EffiEval vs. TailoredBench Spearman ρ ($\uparrow$) and MAE ($\downarrow$) averaged across target-model evaluations. **Bold** marks the best and underline the second best per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
ARC-Challenge	EFFIEVAL	0.7520	0.0902	0.8230	0.0638	<u>0.8640</u>	0.0524	<u>0.8900</u>	<u>0.0465</u>	<u>0.9180</u>	0.0370	0.8494	0.0580
	TAILOREDBENCH	<u>0.7110</u>	0.0717	0.7460	0.0705	0.7540	0.0706	0.7550	0.0728	0.7530	0.0713	0.7438	0.0714
	REPCORE	<u>0.7070</u>	<u>0.0720</u>	<u>0.8200</u>	0.0500	0.8710	0.0410	0.9000	0.0360	0.9210	0.0320	<u>0.8438</u>	0.0462
BBH	EFFIEVAL	0.4590	0.1390	0.5490	0.1130	0.6550	0.1030	0.7100	0.0915	0.7650	0.0824	0.6276	0.1058
	TAILOREDBENCH	<u>0.6870</u>	<u>0.0990</u>	<u>0.7590</u>	<u>0.0795</u>	<u>0.7850</u>	<u>0.0748</u>	<u>0.7830</u>	<u>0.0743</u>	<u>0.7850</u>	<u>0.0737</u>	0.7598	0.0803
	REPCORE	0.7180	0.0950	0.8240	0.0690	0.8700	0.0570	0.8980	0.0490	0.9130	0.0450	0.8446	0.0630
GSM8K	EFFIEVAL	0.7090	0.0973	<u>0.7970</u>	0.0831	0.8350	0.0932	<u>0.8640</u>	0.0950	<u>0.8790</u>	0.0856	0.8168	0.0908
	TAILOREDBENCH	<u>0.7410</u>	0.0644	<u>0.7670</u>	<u>0.0765</u>	<u>0.7750</u>	<u>0.0750</u>	<u>0.7740</u>	<u>0.0789</u>	<u>0.7770</u>	<u>0.0770</u>	0.7668	0.0744
	REPCORE	0.7630	<u>0.0710</u>	0.8720	0.0470	0.9050	0.0390	0.9260	0.0350	0.9380	0.0320	0.8808	0.0448
MMLU-Pro	EFFIEVAL	0.6060	0.2170	0.7380	0.1860	0.7850	0.1790	0.8050	0.1760	0.8010	0.1680	0.7470	0.1852
	TAILOREDBENCH	<u>0.7090</u>	<u>0.0917</u>	<u>0.7810</u>	<u>0.0752</u>	<u>0.8150</u>	<u>0.0677</u>	<u>0.8170</u>	<u>0.0666</u>	<u>0.8220</u>	<u>0.0656</u>	0.7888	0.0734
	REPCORE	0.7320	0.0910	0.8290	0.0670	0.8750	0.0550	0.9060	0.0480	0.9240	0.0430	0.8532	0.0608
SEED-Bench-2-Plus	EFFIEVAL	0.6760	0.1890	0.7760	0.1170	<u>0.8130</u>	0.1010	<u>0.8300</u>	0.0852	<u>0.8570</u>	0.0731	0.7904	0.1131
	TAILOREDBENCH	0.6200	0.0810	0.6570	<u>0.0788</u>	0.6970	<u>0.0737</u>	0.7080	<u>0.0728</u>	0.7060	<u>0.0729</u>	0.6776	<u>0.0758</u>
	REPCORE	<u>0.6420</u>	<u>0.0840</u>	<u>0.7550</u>	0.0600	0.8190	0.0510	0.8530	0.0450	0.8740	0.0400	<u>0.7886</u>	0.0560
Average	EFFIEVAL	0.6404	0.1465	0.7366	0.1126	0.7904	0.1057	0.8198	0.0988	0.8440	0.0892	0.7662	0.1106
	TAILOREDBENCH	<u>0.6936</u>	0.0816	<u>0.7420</u>	<u>0.0761</u>	<u>0.7652</u>	<u>0.0724</u>	<u>0.7674</u>	<u>0.0731</u>	<u>0.7686</u>	<u>0.0721</u>	0.7474	0.0750
	REPCORE	0.7124	<u>0.0826</u>	0.8200	0.0586	0.8680	0.0486	0.8966	0.0426	0.9140	0.0384	0.8422	0.0542

Table 13 compares EffiEval and TailoredBench with REPCORE across all five benchmarks and five coreset budgets. These two methods are included as auxiliary baselines to provide a broader comparison with recent benchmark compression methods, but are not further used in diagnostic ablations because their assumptions can be fragile under the source-scarce setting studied here.

TailoredBench includes an explicit source-model filtering stage before subset construction. In the source-scarce setting,

this stage further narrows the already limited source pool, leaving the method with less usable source information than other baselines. EffiEval, on the other hand, relies on the transferability of activation-based similarity across models, whose reliability can fluctuate considerably in heterogeneous source pools with different architectures and model families. These source-scarce failure modes are also reflected in their limited competitiveness in Table 13. Therefore, to control for factors unrelated to the diagnostic variables under study, we report their full end-to-end results here but do not include them in the subsequent ablation analyses.

C.4. Stability Comparison: REPCORE vs. Baselines

In this section, we further examine the stability of different compression methods by reporting the Combo Std across 10 random source-model combinations, with each combination averaged over 50 random anchor selections. Figures 4 and 5 visualize the mean performance and standard deviation on all five benchmarks, split into two parts for readability. In each benchmark, the left panel reports Spearman’s rank correlation and the right panel reports MAE across coreset sizes. The error bars indicate variation across runs. Overall, REPCORE (red star) achieves consistently strong performance with relatively small variance compared to baselines, providing further evidence for the effectiveness and robustness of our method.

C.5. Computational Cost and Storage Overhead

In this section, we report wall-clock timing and storage measurements under the same model pool and experimental configuration as the main experiments to quantify the additional cost introduced by REPCORE. Following the structure of the pipeline, we decompose the cost into three parts: running each source model on the full benchmark, applying each method’s compression procedure after source inference, and storing the hidden states used by REPCORE. The first part is common to all methods that rely on source models, since they must evaluate the source models on the full benchmark to construct the response matrix for coreset selection and score extrapolation. REPCORE reuses the same source inference runs. Its only modification during this stage is to register a forward hook that records hidden states during the forward pass. All timing measurements in this section are obtained on a server equipped with NVIDIA H20 96GB GPUs.

Source-model inference cost. We measure the average wall-clock time for a single source model to complete inference on each benchmark, with and without registering the forward hook used to capture hidden states. As shown in Table 14, the hook introduces no meaningful additional overhead: the average runtime increases from 9852.8s to 9892.5s, resulting approximately 0.4% relative overhead.

Table 14. Source-model inference time with and without hidden-state extraction. The numbers report the average wall-clock time for a single source model to complete inference on one benchmark.

	Hook (s)	No hook (s)
Average	9892.5	9852.8

REPCORE compression cost after source inference. After source-model outputs and hidden states have been collected, REPCORE runs its compression procedure, including representation alignment, consensus embedding construction, clustering, and fitting the lightweight extrapolator. This stage takes 479.7s on average over the five benchmarks.

Hidden-state storage. The additional storage required by REPCORE comes from saving source-model hidden states. As shown in Table 15, the storage remains modest across benchmarks, ranging from 0.11GB on ARC-Challenge to 1.25GB on MMLU-Pro.

Table 15. Storage overhead for saved hidden states.

Dataset	ARC-C	BBH	GSM8K	MMLU-Pro	SEED-Bench
Size (GB)	0.11	0.67	0.13	1.25	0.17

C.6. Discrete-Representation Ablation

Table 16 presents a controlled ablation that isolates the contribution of continuous aligned representations in the coreset selection stage. We replace our consensus embeddings with concatenated binary correctness vectors and perform clustering

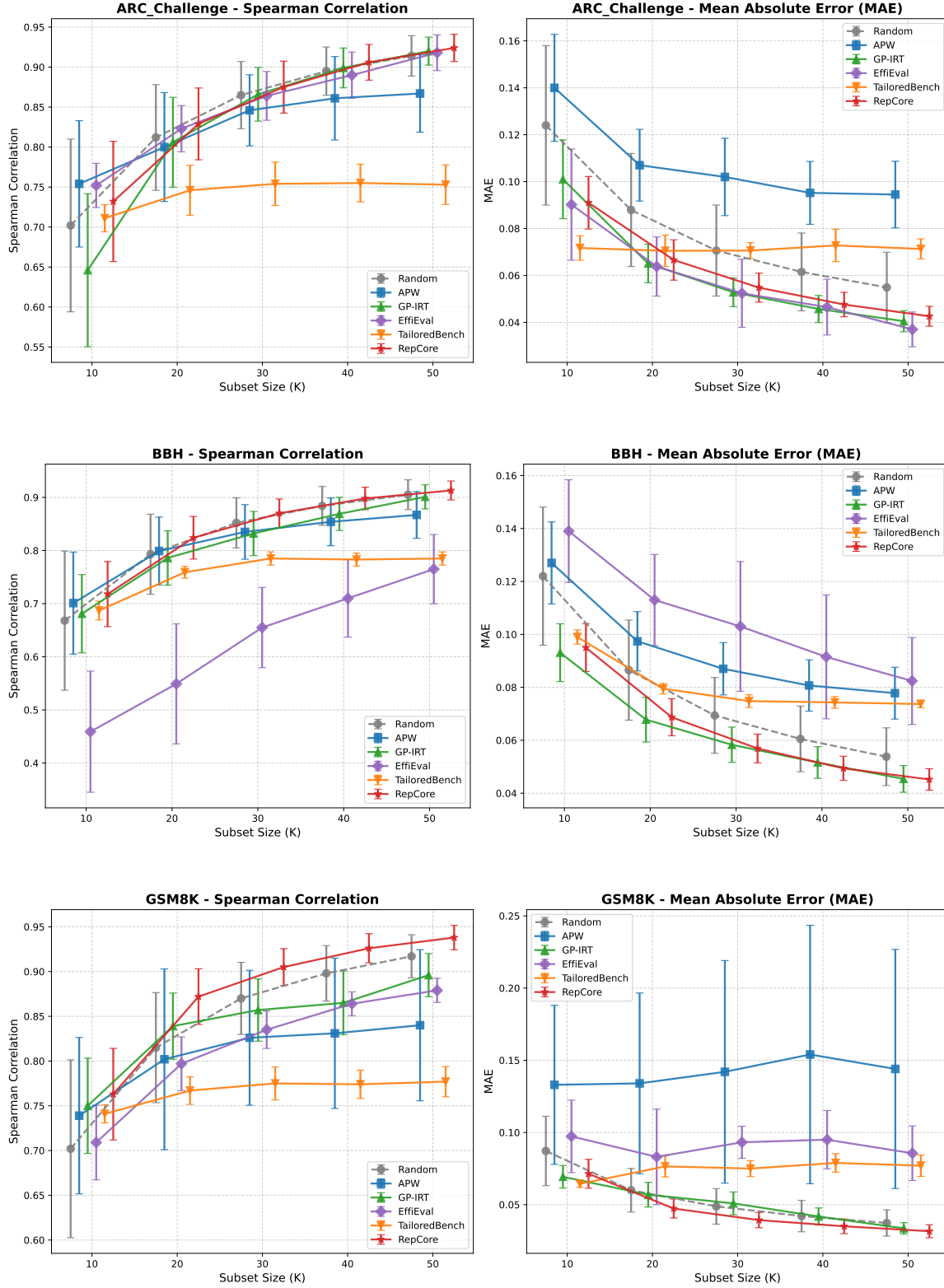


Figure 4. Stability Analysis (Part I): ARC, BBH, and GSM8K. The charts display Spearman Correlation (left) and MAE (right) with standard deviation error bars. Note the stability of REPCORE across varying coresets sizes.

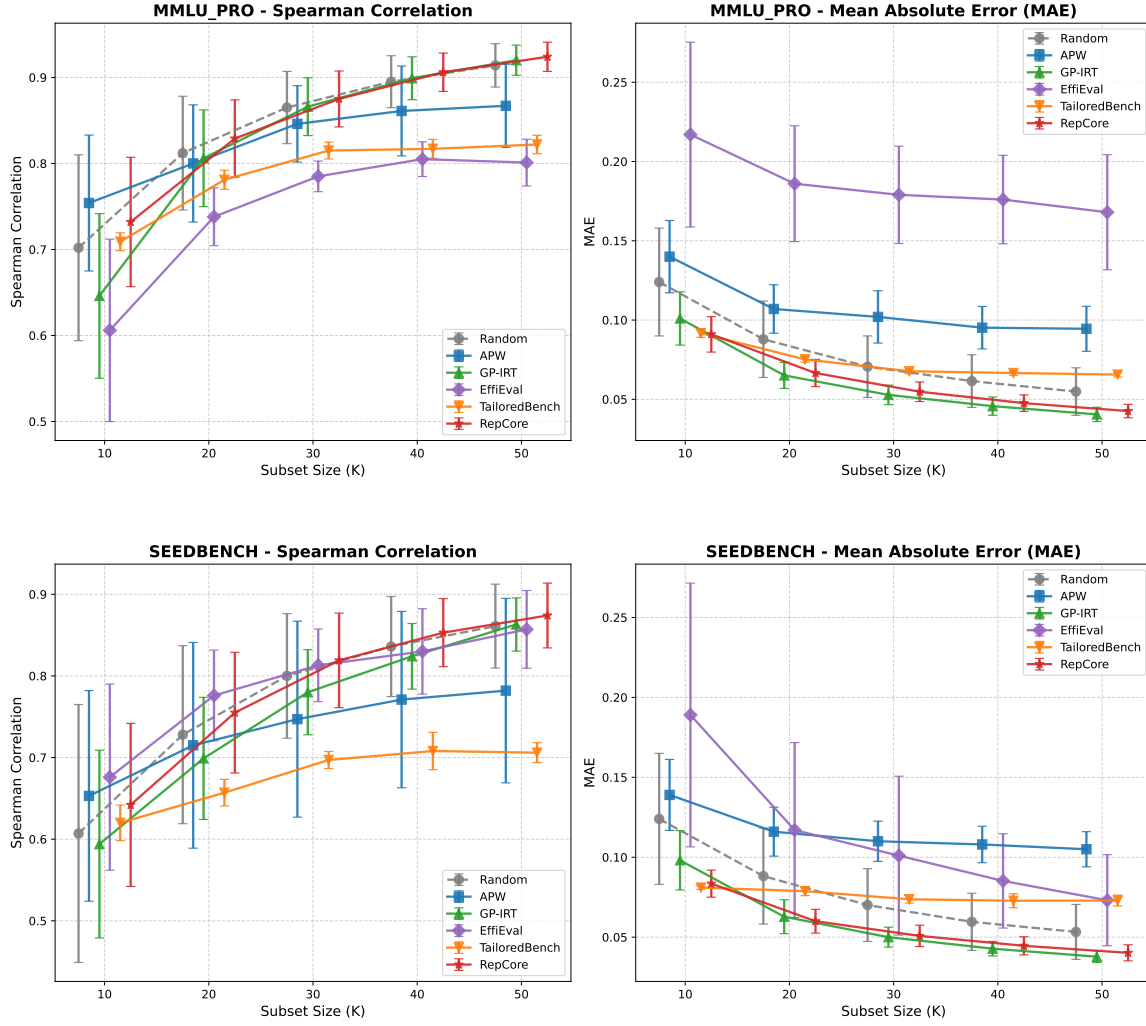


Figure 5. Stability Analysis (Part II): MMLU-PRO and SEEDBENCH. Comparisons on larger-scale benchmarks similarly demonstrate the robustness of the proposed method.

in this discrete space (0/1 K-MEANS+RIDGE), while keeping the downstream extrapolation stage identical. This design ensures that any performance difference is attributable to the representation used for summarizing item structure rather than the predictor itself.

Across all datasets, REPCORE consistently yields higher (or comparable) Spearman correlation and lower MAE for most budgets. While the discrete baseline can be competitive on MAE at the smallest budget for a few cases, REPCORE achieves more stable improvements as K increases, suggesting that the continuous embedding geometry captures fine-grained inter-item relations that binary outcome vectors fail to preserve, thereby enabling more reliable and transferable coreset selection.

Table 16. Discrete-representation ablation. Per-dataset results comparing REPCORE with a discrete clustering variant (0/1 K-MEANS+RIDGE). We report Spearman’s ρ ($\uparrow$) and MAE ($\downarrow$) across budgets $K \in \{10, 20, 30, 40, 50\}$. **Bold** indicates the best result.

Benchmark	Method	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$	
		$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
ARC-Challenge	0/1 K-MEANS+RIDGE	0.685	0.0719	0.803	0.0578	0.850	0.0512	0.874	0.0485	0.894	0.0449
	REPCORE	0.707	0.0720	0.820	0.0500	0.871	0.0409	0.900	0.0358	0.921	0.0321
BBH	0/1 K-MEANS+RIDGE	0.717	0.0958	0.821	0.0713	0.862	0.0604	0.884	0.0533	0.896	0.0487
	REPCORE	0.718	0.0950	0.824	0.0687	0.870	0.0569	0.898	0.0494	0.913	0.0452
GSM8K	0/1 K-MEANS+RIDGE	0.754	0.0745	0.845	0.0579	0.886	0.0514	0.909	0.0471	0.922	0.0442
	REPCORE	0.763	0.0714	0.872	0.0473	0.905	0.0393	0.926	0.0350	0.938	0.0316
SEED-Bench-2-Plus	0/1 K-MEANS+RIDGE	0.514	0.0860	0.697	0.0656	0.786	0.0566	0.829	0.0511	0.857	0.0473
	REPCORE	0.642	0.0835	0.755	0.0600	0.819	0.0508	0.853	0.0446	0.874	0.0402
MMLU-Pro	0/1 K-MEANS+RIDGE	0.725	0.0874	0.824	0.0685	0.865	0.0575	0.890	0.0505	0.904	0.0462
	REPCORE	0.732	0.0910	0.829	0.0666	0.875	0.0548	0.906	0.0476	0.924	0.0426

C.7. Sensitivity to MLP Architecture

To examine whether REPCORE is sensitive to the specific width and depth of the shared MLP backbone, we conduct an architecture ablation on BBH. We compare the default backbone with wider, deeper, and shallower variants, with all other components kept unchanged. Table 17 reports the results across different coreset budgets.

Table 17. Results on BBH under alternative shared-backbone configurations. Spearman’s ρ ($\uparrow$) and MAE ($\downarrow$) are reported across coreset budgets $K \in 10, 20, 30, 40, 50$. The first row is the original REPCORE setting; the remaining rows are different MLP hidden-layer configurations (values denote hidden sizes). **Bold** marks the best and underline the second best per column.

MLP hidden dims	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
REPCORE (OURS)	0.718	0.095	0.824	0.069	0.870	<u>0.057</u>	<u>0.898</u>	<u>0.049</u>	0.913	0.045	<u>0.845</u>	<u>0.063</u>
1024-512-128-32	0.723	<u>0.095</u>	<u>0.822</u>	0.070	<u>0.868</u>	0.057	0.894	0.050	0.911	0.045	0.844	0.064
1024-512-32	<u>0.719</u>	0.095	0.821	0.070	0.868	0.056	0.900	0.048	0.918	0.043	0.845	0.062
512-32	0.710	0.096	0.816	<u>0.069</u>	0.866	0.057	0.897	0.050	<u>0.917</u>	<u>0.045</u>	0.841	0.063

Overall, the results are highly similar across different MLP architectures, with no significant performance gap among the default, wider, deeper, and shallower variants. This suggests that REPCORE is not sensitive to moderate changes in the width or depth of the shared MLP backbone.

C.8. Extrapolation Feature Ablation

To examine the design choice of using the averaged source correctness x_i as the extrapolation feature, we compare it with an alternative that uses the concatenated source correctness vector as input to the same ridge regressor. The scalar variant, denoted as $\text{label}_{\text{avg}}$, uses $x_i = \frac{1}{|S|} \sum_{m \in S} y_{m,i}$, whereas $\text{label}_{\text{vec}}$ uses $(y_{m_1,i}, \dots, y_{m_{|S|},i}) \in \mathbb{R}^{|S|}$, where $S = \{m_1, \dots, m_{|S|}\}$. All other components are kept unchanged. As shown in Table 18, the scalar feature consistently

improves ranking recovery while maintaining comparable MAE, supporting its use as a stable low-variance item-level anchor in the source-scarce regime.

Table 18. Extrapolation feature ablation averaged over five benchmarks. $\text{label}_{\text{avg}}$ uses the scalar averaged source correctness x_i as the input feature, whereas $\text{label}_{\text{vec}}$ uses the full source correctness vector. Each cell reports Spearman’s ρ / MAE.

Feature	K=10	K=20	K=30	K=40	K=50
$\text{label}_{\text{vec}}$	0.567 / 0.081	0.694 / 0.060	0.788 / 0.050	0.835 / 0.044	0.861 / 0.040
$\text{label}_{\text{avg}}$	0.642 / 0.084	0.755 / 0.060	0.819 / 0.051	0.853 / 0.045	0.874 / 0.040

C.9. Detailed Results on Source Model Quantity

In this section, we provide a comprehensive breakdown of the analysis presented in Section 3. To rigorously evaluate the sensitivity of each method to the availability of source models, we vary the source pool size $|\mathcal{S}| \in \{5, 10, 15, 20\}$ across the full spectrum of computational budgets $K \in \{10, 20, 30, 40, 50\}$. The detailed performance metrics for each budget setting are reported in Tables 19-23.

The results consistently demonstrate the robustness of REPCORE in source-scarce regimes. As evidenced across all budget levels, output-based baselines such as ANCHORPOINTS exhibit sharp performance degradation when $|\mathcal{S}| = 5$, suggesting a heavy reliance on extensive statistical sampling to estimate item difficulty. In contrast, REPCORE maintains high ranking correlations even with minimal supervision, validating our hypothesis that the geometric structure of the representation space can be effectively aligned with limited source signals. Furthermore, performance saturation is observed as $|\mathcal{S}|$ approaches 20, indicating that our framework captures the essential topology of the item space early.

Table 19. Detailed results with budget $K = 10$ under varying source pool sizes $|\mathcal{S}|$. We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$). **Bold** and underlined values denote the best and second-best results, respectively.

Method	ARC-C		BBH		GSM8K		SeedBench		MMLU-Pro	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
$ \mathcal{S} = 5$										
ANCHORPOINTS	<u>0.632</u>	0.216	0.614	0.176	0.714	0.210	0.513	0.228	0.673	0.187
GP-IRT	0.609	<u>0.088</u>	0.639	<u>0.106</u>	<u>0.729</u>	<u>0.079</u>	<u>0.581</u>	<u>0.095</u>	<u>0.703</u>	<u>0.115</u>
REPCORE	0.704	0.077	0.706	0.104	0.749	0.074	0.590	0.088	0.744	0.096
$ \mathcal{S} = 10$										
ANCHORPOINTS	0.724	0.132	<u>0.701</u>	0.127	0.739	0.133	0.653	0.139	0.754	0.140
GP-IRT	0.706	<u>0.076</u>	0.681	0.093	<u>0.750</u>	0.069	0.594	<u>0.098</u>	0.646	<u>0.101</u>
REPCORE	<u>0.707</u>	0.072	0.718	<u>0.095</u>	0.763	<u>0.071</u>	<u>0.642</u>	0.084	<u>0.732</u>	0.091
$ \mathcal{S} = 15$										
ANCHORPOINTS	0.562	0.126	0.765	0.117	0.784	0.110	0.715	0.123	0.798	0.128
GP-IRT	0.719	<u>0.081</u>	<u>0.674</u>	0.090	0.734	<u>0.073</u>	0.607	<u>0.092</u>	0.612	0.098
REPCORE	<u>0.699</u>	0.074	0.672	<u>0.098</u>	<u>0.763</u>	0.070	<u>0.619</u>	0.091	<u>0.687</u>	0.095
$ \mathcal{S} = 20$										
ANCHORPOINTS	0.683	0.106	0.787	0.115	<u>0.762</u>	0.098	0.737	0.124	0.809	0.121
GP-IRT	0.638	<u>0.082</u>	0.620	0.096	0.711	<u>0.070</u>	0.616	<u>0.091</u>	0.566	<u>0.106</u>
REPCORE	<u>0.671</u>	0.074	<u>0.717</u>	<u>0.099</u>	0.787	0.065	<u>0.638</u>	0.086	<u>0.682</u>	0.093

C.10. Detailed Results of Source Composition Ablation

In this section, we provide the detailed breakdown of the source model composition ablation study across five datasets: ARC-Challenge, BBH, GSM8K, SeedBench, and MMLU-Pro. Table 24 presents the Ranking Correlation (Spearman ρ) and Estimation Error (MAE) across coreset budgets $K \in \{10, 20, 30, 40, 50\}$.

Table 20. Detailed results with budget $K = 20$ under varying source pool sizes $|S|$. We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$). **Bold** and underlined values denote the best and second-best results, respectively.

Method	ARC-C		BBH		GSM8K		SeedBench		MMLU-Pro	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
$ S = 5$										
ANCHORPOINTS	0.647	0.260	0.619	0.156	0.722	0.230	0.539	0.220	0.678	0.172
GP-IRT	<u>0.726</u>	<u>0.077</u>	<u>0.730</u>	<u>0.078</u>	<u>0.820</u>	<u>0.053</u>	<u>0.683</u>	<u>0.071</u>	<u>0.808</u>	0.067
REPCORE	0.818	0.053	0.808	0.075	0.845	0.051	0.731	0.065	0.830	<u>0.069</u>
$ S = 10$										
ANCHORPOINTS	0.767	0.136	<u>0.799</u>	0.097	0.794	0.132	<u>0.741</u>	0.119	<u>0.800</u>	0.107
GP-IRT	0.808	<u>0.057</u>	<u>0.786</u>	0.068	0.839	<u>0.057</u>	0.699	<u>0.063</u>	<u>0.806</u>	0.065
REPCORE	0.820	0.050	0.824	<u>0.069</u>	0.872	0.047	0.755	0.060	0.829	<u>0.067</u>
$ S = 15$										
ANCHORPOINTS	0.813	0.107	0.840	0.086	<u>0.846</u>	0.102	0.796	0.096	0.848	0.094
GP-IRT	0.826	<u>0.054</u>	<u>0.779</u>	0.068	0.828	<u>0.051</u>	<u>0.719</u>	0.061	0.755	<u>0.069</u>
REPCORE	<u>0.822</u>	0.051	<u>0.807</u>	<u>0.072</u>	0.856	0.050	0.713	<u>0.063</u>	<u>0.838</u>	0.067
$ S = 20$										
ANCHORPOINTS	<u>0.817</u>	0.092	0.861	0.084	0.831	0.088	0.820	0.092	0.861	0.088
GP-IRT	0.774	<u>0.055</u>	0.775	0.067	0.842	0.049	0.716	0.059	0.687	0.078
REPCORE	0.823	0.051	<u>0.844</u>	<u>0.071</u>	0.870	<u>0.051</u>	<u>0.743</u>	<u>0.061</u>	<u>0.827</u>	0.065

Table 21. Detailed results with budget $K = 30$ under varying source pool sizes $|S|$. We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$). **Bold** and underlined values denote the best and second-best results, respectively.

Method	ARC-C		BBH		GSM8K		SeedBench		MMLU-Pro	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
$ S = 5$										
ANCHORPOINTS	0.656	0.259	0.627	0.155	0.718	0.260	0.535	0.212	0.664	0.163
GP-IRT	<u>0.813</u>	<u>0.068</u>	<u>0.806</u>	<u>0.062</u>	0.864	0.042	<u>0.774</u>	<u>0.055</u>	<u>0.853</u>	0.057
REPCORE	0.868	0.044	0.860	0.060	0.900	<u>0.044</u>	0.790	0.053	0.882	0.057
$ S = 10$										
ANCHORPOINTS	0.792	0.138	<u>0.835</u>	0.087	0.832	0.125	<u>0.794</u>	0.108	0.846	0.102
GP-IRT	<u>0.845</u>	<u>0.052</u>	<u>0.832</u>	<u>0.058</u>	<u>0.857</u>	<u>0.051</u>	0.780	0.050	<u>0.866</u>	0.053
REPCORE	0.871	0.041	0.870	0.057	0.905	0.039	0.819	<u>0.051</u>	0.875	<u>0.055</u>
$ S = 15$										
ANCHORPOINTS	0.822	0.104	0.873	0.076	<u>0.887</u>	0.098	0.838	0.085	<u>0.873</u>	0.084
GP-IRT	0.877	0.040	0.843	0.056	0.872	<u>0.043</u>	<u>0.787</u>	0.048	0.837	<u>0.056</u>
REPCORE	<u>0.875</u>	<u>0.041</u>	<u>0.857</u>	<u>0.058</u>	0.898	0.041	0.783	<u>0.052</u>	0.886	0.054
$ S = 20$										
ANCHORPOINTS	<u>0.835</u>	0.094	0.886	0.074	0.874	0.084	0.864	0.076	0.889	0.079
GP-IRT	0.833	<u>0.043</u>	0.858	0.054	<u>0.882</u>	<u>0.042</u>	0.768	0.049	0.798	<u>0.063</u>
REPCORE	0.878	0.041	<u>0.880</u>	<u>0.055</u>	0.909	0.041	<u>0.799</u>	<u>0.050</u>	<u>0.880</u>	0.053

Table 22. Detailed results with budget $K = 40$ under varying source pool sizes $|\mathcal{S}|$. We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$). **Bold** and underlined values denote the best and second-best results, respectively.

Method	ARC-C		BBH		GSM8K		SeedBench		MMLU-Pro	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
$ \mathcal{S} = 5$										
ANCHORPOINTS	0.650	0.269	0.618	<u>0.150</u>	0.714	0.227	0.539	<u>0.215</u>	0.413	0.182
GP-IRT	<u>0.817</u>	<u>0.058</u>	<u>0.853</u>	0.054	0.892	0.037	0.829	0.047	0.885	0.050
REPCORE	0.894	0.039	0.887	0.054	0.922	<u>0.040</u>	0.834	0.047	0.906	<u>0.051</u>
$ \mathcal{S} = 10$										
ANCHORPOINTS	0.797	0.145	0.854	0.081	0.864	0.117	0.818	0.103	0.861	0.095
GP-IRT	0.866	<u>0.043</u>	0.869	<u>0.052</u>	<u>0.865</u>	<u>0.042</u>	<u>0.824</u>	0.043	0.899	0.046
REPCORE	0.900	0.036	0.898	0.049	0.926	0.035	0.853	<u>0.045</u>	0.906	<u>0.048</u>
$ \mathcal{S} = 15$										
ANCHORPOINTS	0.843	0.102	0.896	0.068	0.903	0.090	0.865	0.082	0.894	0.081
GP-IRT	<u>0.901</u>	0.035	0.887	0.047	0.897	0.036	<u>0.841</u>	0.041	0.887	0.046
REPCORE	0.903	<u>0.036</u>	<u>0.894</u>	<u>0.050</u>	0.922	0.036	0.822	<u>0.045</u>	0.908	<u>0.047</u>
$ \mathcal{S} = 20$										
ANCHORPOINTS	0.767	0.096	0.906	0.065	0.892	0.080	0.883	0.070	<u>0.904</u>	0.076
GP-IRT	0.867	<u>0.037</u>	0.894	0.046	0.900	0.038	0.813	0.042	<u>0.864</u>	<u>0.052</u>
REPCORE	0.903	0.035	0.911	<u>0.047</u>	0.928	0.037	<u>0.831</u>	<u>0.045</u>	0.906	0.047

Table 23. Detailed results with budget $K = 50$ under varying source pool sizes $|\mathcal{S}|$. We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$). **Bold** and underlined values denote the best and second-best results, respectively.

Method	ARC-C		BBH		GSM8K		SeedBench		MMLU-Pro	
	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$	$\rho \uparrow$	MAE $\downarrow$
$ \mathcal{S} = 5$										
ANCHORPOINTS	0.636	0.255	0.622	0.151	0.718	0.246	0.498	0.215	0.647	0.162
GP-IRT	<u>0.819</u>	<u>0.047</u>	<u>0.880</u>	0.049	0.913	0.032	0.863	0.042	<u>0.906</u>	0.045
REPCORE	0.917	0.035	0.903	<u>0.050</u>	0.936	<u>0.036</u>	0.863	<u>0.043</u>	0.922	<u>0.046</u>
$ \mathcal{S} = 10$										
ANCHORPOINTS	0.796	0.143	0.867	0.078	0.880	0.116	0.835	0.098	0.867	0.094
GP-IRT	0.884	<u>0.036</u>	<u>0.901</u>	0.045	0.896	0.034	0.863	0.038	<u>0.920</u>	0.041
REPCORE	0.921	0.032	0.913	0.045	0.938	0.032	0.874	<u>0.040</u>	0.924	<u>0.043</u>
$ \mathcal{S} = 15$										
ANCHORPOINTS	0.830	0.118	0.904	0.065	0.913	0.089	0.877	0.078	0.906	0.079
GP-IRT	<u>0.919</u>	0.030	0.911	0.042	<u>0.916</u>	0.031	<u>0.870</u>	0.037	<u>0.912</u>	0.041
REPCORE	0.921	<u>0.032</u>	0.911	<u>0.045</u>	0.936	<u>0.032</u>	0.849	<u>0.041</u>	0.926	<u>0.042</u>
$ \mathcal{S} = 20$										
ANCHORPOINTS	0.850	0.087	<u>0.919</u>	0.059	0.904	0.079	0.893	0.066	<u>0.916</u>	0.074
GP-IRT	<u>0.898</u>	0.032	0.915	0.041	<u>0.910</u>	<u>0.034</u>	0.852	0.036	0.896	<u>0.045</u>
REPCORE	0.921	0.032	0.927	<u>0.043</u>	0.941	0.033	<u>0.859</u>	<u>0.040</u>	0.924	0.041

Table 24. Detailed ablation study on source model composition across five datasets. We compare single model families (e.g., Qwen, Llama, Ovis, InternVL) against a **Diverse** source composition (Ours). We report Spearman’s rank correlation (ρ , $\uparrow$) and Mean Absolute Error (MAE, $\downarrow$). **Bold** and underlined values denote the best and second-best results, respectively.

(a) ARC-Challenge										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Qwen	0.658	0.800	0.853	0.888	0.908	0.081	<u>0.055</u>	<u>0.046</u>	<u>0.041</u>	<u>0.037</u>
Llama	0.725	0.833	0.879	0.904	<u>0.921</u>	<u>0.078</u>	0.056	0.047	0.042	0.038
Diverse (Ours)	<u>0.707</u>	<u>0.820</u>	<u>0.871</u>	<u>0.900</u>	0.921	0.072	0.050	0.041	0.036	0.032

(b) BBH										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Qwen	0.651	0.759	0.814	0.842	0.862	0.119	0.085	0.068	0.060	0.054
Llama	<u>0.661</u>	<u>0.811</u>	<u>0.862</u>	<u>0.888</u>	<u>0.904</u>	<u>0.099</u>	<u>0.072</u>	<u>0.060</u>	<u>0.054</u>	<u>0.048</u>
Diverse (Ours)	0.718	0.824	0.870	0.898	0.913	0.095	0.069	0.057	0.049	0.045

(c) GSM8K										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Qwen	0.739	0.832	0.885	0.913	0.928	0.090	<u>0.065</u>	<u>0.057</u>	<u>0.052</u>	<u>0.048</u>
Llama	0.754	0.854	0.898	0.919	0.934	<u>0.087</u>	0.067	0.060	0.054	0.050
Diverse (Ours)	0.763	0.872	0.905	0.926	0.938	0.071	0.047	0.039	0.035	0.032

(d) SeedBench										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Ovis	0.597	0.756	0.826	0.864	0.883	<u>0.093</u>	<u>0.063</u>	<u>0.054</u>	<u>0.049</u>	<u>0.045</u>
InternVL	<u>0.618</u>	0.741	0.802	0.837	0.859	0.094	0.073	0.065	0.059	0.055
Diverse (Ours)	0.642	<u>0.755</u>	<u>0.819</u>	<u>0.853</u>	<u>0.874</u>	0.084	0.060	0.051	0.045	0.040

(e) MMLU-Pro										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Qwen	0.703	0.816	0.860	0.888	0.903	0.101	0.074	0.059	0.052	0.047
Llama	<u>0.716</u>	<u>0.823</u>	0.876	<u>0.902</u>	<u>0.921</u>	<u>0.097</u>	<u>0.070</u>	<u>0.057</u>	<u>0.050</u>	<u>0.045</u>
Diverse (Ours)	0.732	0.829	<u>0.875</u>	0.906	0.924	0.091	0.067	0.055	0.048	0.043

C.11. Detailed Results of Source Capability Range Ablation

In this section, we present the complete experimental results analyzing the impact of source model capability on compression performance. Table 25 provides a dataset-wise breakdown of the Ranking Correlation (Spearman ρ) and Estimation Error (MAE) across coreset budgets $K \in \{10, 20, 30, 40, 50\}$.

We compare three distinct source composition strategies:

- **Strong:** Pools consisting exclusively of the top-10 strongest models.
- **Weak:** Pools consisting exclusively of the bottom-10 weakest models.
- **Diverse (Ours):** A heterogeneous mix spanning the full capability spectrum.

The detailed results reaffirm the analysis in the main text: The **Diverse** strategy consistently achieves the best trade-off between ranking accuracy and estimation stability across all five datasets.

C.12. Detailed Results on Coreset Size Generalization

In this section, we provide the comprehensive numerical results for the coreset size generalization experiments. We evaluate the robustness of different selection methods across three coreset sizes (100, 150, and 200 points). Table 26 presents the detailed breakdown, with Panel A showing the Spearman’s rank correlation (ρ) and Panel B showing the Mean Absolute Error (MAE) on five benchmarks.

C.13. Robustness to Temporal Drift

This section reports the complete results for the temporal distribution shift setting described in Section 5. Table 27 summarizes Spearman’s ρ and MAE across coreset budgets $K \in \{10, 20, 30, 40, 50\}$ on five benchmarks. The results are consistent with the trend observed in the main text.

C.14. Source Hidden-State Masking Analysis

This section details the mixed-source simulation used in Section 6.1. We first describe the pseudo-representation construction for masked sources and the two masking methods, then report the full masking results together with additional analyses of robustness and sensitivity.

C.14.1. PSEUDO-REPRESENTATION CONSTRUCTION.

Let O denote the sources with accessible hidden states and M denote the sources whose hidden states are masked, with $S = O \cup M$. For each masked source $m \in M$, we approximate its item representation using accessible sources with similar output behavior. Specifically, we compute the output-behavior similarity and normalize it as

$$\text{sim}(m, o) = \cos(y_m, y_o), \quad \alpha_{m,o} = \text{softmax}_{o \in O}(\text{sim}(m, o)), \quad (21)$$

where y_m and y_o are binary correctness vectors over the full benchmark. We then construct the pseudo-representation of masked source m on item i as

$$\hat{z}_{m,i} = \sum_{o \in O} \alpha_{m,o} \cdot z_{o,i}. \quad (22)$$

The consensus embedding used for clustering combines real aligned embeddings from accessible sources with pseudo-representations for masked sources:

$$e_i = \frac{1}{|S|} \left(\sum_{o \in O} z_{o,i} + \sum_{m \in M} \hat{z}_{m,i} \right). \quad (23)$$

This replacement is applied only during coreset selection, while the extrapolation stage remains unchanged. Thus, the pseudo-representation is a practical approximation for partially accessible source pools rather than a substitute for true hidden-state access.

Table 25. Detailed ablation study on source capability range across five datasets. We compare homogeneous source pools consisting of only **Strong** or **Weak** models against a **Diverse** source composition (Ours). We report Spearman’s rank correlation (ρ) and Mean Absolute Error (MAE). **Bold** and underlined values denote the best and second-best results, respectively.

(a) ARC-Challenge										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Strong	0.479	0.659	<u>0.785</u>	0.843	0.868	0.125	0.144	0.154	0.144	0.128
Weak	<u>0.536</u>	<u>0.713</u>	0.782	<u>0.844</u>	<u>0.873</u>	<u>0.092</u>	<u>0.055</u>	<u>0.043</u>	<u>0.037</u>	0.032
Diverse (Ours)	0.707	0.820	0.871	0.900	0.921	0.072	0.050	0.041	0.036	<u>0.032</u>

(b) BBH										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Strong	0.502	0.705	0.770	0.785	0.823	0.148	0.116	0.073	0.066	0.062
Weak	<u>0.654</u>	<u>0.758</u>	<u>0.817</u>	<u>0.855</u>	<u>0.890</u>	<u>0.098</u>	<u>0.075</u>	<u>0.061</u>	<u>0.054</u>	<u>0.050</u>
Diverse (Ours)	0.718	0.824	0.870	0.898	0.913	0.095	0.069	0.057	0.049	0.045

(c) GSM8K										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Strong	0.718	0.812	<u>0.858</u>	<u>0.888</u>	<u>0.908</u>	0.120	0.094	0.079	0.065	0.056
Weak	0.612	<u>0.817</u>	0.839	0.874	0.905	<u>0.074</u>	<u>0.049</u>	<u>0.040</u>	0.035	0.030
Diverse (Ours)	0.763	0.872	0.905	0.926	0.938	0.071	0.047	0.039	<u>0.035</u>	<u>0.032</u>

(d) SeedBench										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Strong	0.626	0.694	<u>0.748</u>	<u>0.802</u>	<u>0.846</u>	<u>0.089</u>	<u>0.079</u>	0.073	0.063	0.057
Weak	0.683	<u>0.696</u>	0.741	0.793	0.833	0.114	0.083	<u>0.063</u>	<u>0.052</u>	<u>0.051</u>
Diverse (Ours)	<u>0.642</u>	0.755	0.819	0.853	0.874	0.084	0.060	0.051	0.045	0.040

(e) MMLU-Pro										
Source Pool	$\rho \uparrow$					MAE $\downarrow$				
	K=10	K=20	K=30	K=40	K=50	K=10	K=20	K=30	K=40	K=50
Strong	0.482	0.544	0.728	0.776	0.800	<u>0.119</u>	0.094	0.079	0.081	0.070
Weak	<u>0.614</u>	<u>0.743</u>	<u>0.815</u>	<u>0.871</u>	<u>0.896</u>	0.119	<u>0.082</u>	<u>0.067</u>	<u>0.056</u>	<u>0.049</u>
Diverse (Ours)	0.732	0.829	0.875	0.906	0.924	0.091	0.067	0.055	0.048	0.043

Table 26. Detailed generalization results with varying coreset sizes. We report Spearman’s ρ and MAE on five benchmarks. The best results are **bolded** and the second best are underlined.

Size	Method	ARC	BBH	GSM8K	MMLU	SeedBench
Panel A: ρ ($\uparrow$)						
100	RANDOM	0.953	<u>0.948</u>	<u>0.958</u>	0.953	0.921
	APW	0.791	0.883	0.838	0.881	0.770
	GP-IRT	<u>0.957</u>	<u>0.948</u>	0.955	0.960	0.934
	REPCORE	0.960	0.954	0.967	<u>0.959</u>	<u>0.928</u>
150	RANDOM	0.969	<u>0.964</u>	<u>0.972</u>	0.968	0.947
	APW	0.777	0.888	0.829	0.884	0.781
	GP-IRT	<u>0.972</u>	0.961	0.968	0.972	0.957
	REPCORE	0.973	0.967	0.977	<u>0.971</u>	<u>0.948</u>
200	RANDOM	0.977	<u>0.972</u>	<u>0.979</u>	0.975	<u>0.959</u>
	APW	0.784	0.890	0.824	0.887	0.796
	GP-IRT	<u>0.979</u>	0.969	0.973	<u>0.977</u>	0.967
	REPCORE	0.981	0.973	0.982	0.978	<u>0.959</u>
Panel B: MAE ($\downarrow$)						
100	RANDOM	0.0267	0.0378	0.0261	0.0385	0.0377
	APW	0.1430	0.0716	0.1480	0.0885	0.1040
	GP-IRT	<u>0.0228</u>	0.0318	<u>0.0237</u>	0.0282	0.0274
	REPCORE	0.0227	<u>0.0328</u>	0.0233	<u>0.0314</u>	<u>0.0288</u>
150	RANDOM	0.0211	0.0306	<u>0.0206</u>	0.0312	0.0303
	APW	0.1350	0.0704	0.1460	0.0866	0.0953
	GP-IRT	<u>0.0202</u>	<u>0.0289</u>	0.0215	0.0236	0.0222
	REPCORE	0.0189	0.0271	0.0194	<u>0.0261</u>	<u>0.0239</u>
200	RANDOM	<u>0.0175</u>	<u>0.0264</u>	<u>0.0176</u>	0.0271	0.0255
	APW	0.1210	0.0685	0.1330	0.0855	0.0850
	GP-IRT	0.0189	0.0270	0.0204	0.0213	0.0194
	REPCORE	0.0168	0.0239	0.0173	<u>0.0231</u>	<u>0.0216</u>

Table 27. Complete results under temporal distribution shift. Spearman’s ρ ($\uparrow$) and MAE ($\downarrow$) are reported across coreset budgets $K \in \{10, 20, 30, 40, 50\}$. The best results are **bolded** and the second best are underlined.

K	Method	ARC	BBH	GSM8K	MMLU Pro	SeedBench
Panel A: ρ ($\uparrow$)						
10	RANDOM	0.676	0.706	0.714	0.735	0.555
	APW	0.705	<u>0.716</u>	<u>0.758</u>	0.707	0.577
	GP-IRT	<u>0.698</u>	0.704	0.724	<u>0.725</u>	0.549
	REPCORE	0.673	0.719	0.772	0.716	<u>0.574</u>
20	RANDOM	0.791	<u>0.817</u>	0.822	0.842	<u>0.674</u>
	APW	0.779	0.774	0.815	0.803	0.648
	GP-IRT	0.822	0.786	<u>0.826</u>	0.825	0.679
	REPCORE	<u>0.800</u>	0.844	0.870	<u>0.831</u>	0.666
30	RANDOM	0.848	<u>0.868</u>	<u>0.870</u>	0.890	0.747
	APW	0.806	0.819	0.838	0.830	0.691
	GP-IRT	0.865	0.845	0.863	0.870	0.733
	REPCORE	<u>0.861</u>	0.885	0.908	<u>0.878</u>	<u>0.735</u>
40	RANDOM	0.880	<u>0.896</u>	<u>0.898</u>	0.912	0.792
	APW	0.820	<u>0.827</u>	<u>0.855</u>	0.843	0.728
	GP-IRT	<u>0.886</u>	0.886	0.893	0.899	<u>0.791</u>
	REPCORE	0.896	0.909	0.927	<u>0.905</u>	0.779
50	RANDOM	0.901	0.914	<u>0.917</u>	0.928	<u>0.820</u>
	APW	0.820	0.836	0.858	0.867	0.772
	GP-IRT	<u>0.908</u>	0.906	0.916	0.922	0.828
	REPCORE	0.914	0.920	0.942	<u>0.923</u>	0.812
Panel B: MAE ($\downarrow$)						
10	RANDOM	0.0832	0.1180	0.0833	0.1220	0.1180
	APW	0.1140	0.1400	0.1250	0.1420	0.1220
	GP-IRT	0.0671	<u>0.1060</u>	0.0711	<u>0.1030</u>	<u>0.0934</u>
	REPCORE	<u>0.0742</u>	0.0994	<u>0.0724</u>	0.0993	0.0912
20	RANDOM	0.0591	0.0837	0.0576	0.0865	0.0840
	APW	0.1100	0.1180	0.1250	0.1190	0.0992
	GP-IRT	0.0476	0.0703	<u>0.0519</u>	<u>0.0711</u>	0.0614
	REPCORE	<u>0.0479</u>	<u>0.0726</u>	0.0516	0.0694	<u>0.0648</u>
30	RANDOM	0.0484	0.0690	0.0469	0.0704	0.0671
	APW	0.1130	0.1070	0.1290	0.1120	0.0912
	GP-IRT	<u>0.0398</u>	<u>0.0592</u>	<u>0.0434</u>	0.0565	0.0477
	REPCORE	0.0386	0.0581	0.0414	<u>0.0574</u>	<u>0.0525</u>
40	RANDOM	0.0418	0.0587	0.0406	0.0610	0.0573
	APW	0.1130	0.1030	0.1270	0.1080	0.0863
	GP-IRT	<u>0.0352</u>	0.0500	0.0357	0.0492	0.0400
	REPCORE	0.0325	<u>0.0507</u>	<u>0.0366</u>	<u>0.0498</u>	<u>0.0460</u>
50	RANDOM	0.0368	0.0525	0.0356	0.0550	0.0537
	APW	0.1110	0.1000	0.1220	0.0945	0.0854
	GP-IRT	<u>0.0312</u>	0.0441	0.0307	0.0429	0.0361
	REPCORE	0.0296	<u>0.0461</u>	<u>0.0326</u>	<u>0.0453</u>	<u>0.0411</u>

C.14.2. MASKING METHODS.

We consider two masking methods: Random Mask removes source hidden states uniformly at random and averages over 10 masking draws; Advdist Mask instead targets the sources that are least redundant with the rest of the pool, providing a stricter stress test. For each source model s , we compute its behavioral distinctness score as

$$d(s) = \frac{1}{|S| - 1} \sum_{s' \neq s} (1 - \cos(y_s, y_{s'})), \quad (24)$$

where y_s denotes the binary correctness vector of source model s over the full benchmark. Advdist Mask removes sources in descending order of $d(s)$, so the masked sources are expected to be the hardest to approximate from the remaining accessible sources.

C.14.3. FULL MASKING RESULTS ANALYSIS

This part provides the full breakdown of the hidden state unavailable source experiments. Tables 28–32 report mean performance under masking ratios from 10% to 50%, covering five benchmarks and coreset budgets $K \in \{10, 20, 30, 40, 50\}$. Each table reports Spearman’s ρ and MAE averaged over 10 source-model combinations and 50 random anchor selections. For RANDOM PSEUDO, results are further averaged over 10 stochastic masking draws; REPCORE and ADVDIST PSEUDO are deterministic under each source-model combination. The BEST BASELINE row denotes the strongest output-based baseline under the corresponding setting.

Tables 33–37 report $Combo_{std}$, measuring variation across the 10 source-model combinations. For RANDOM PSEUDO, each combination-level score is first averaged over 10 random masking draws before computing $Combo_{std}$. Since this additional averaging structurally reduces variance, its standard deviations are reported as reference values. In contrast, REPCORE and ADVDIST PSEUDO are deterministic for each source-model combination, so their $Combo_{std}$ values are directly comparable. Table 38 further reports $Draw_{std}$ for RANDOM PSEUDO, isolating the sensitivity to which source models are randomly masked.

Random masking remains competitive. Under random masking, the pseudo-representation strategy degrades gradually as the masking ratio increases. Across 125 breakdown cells, corresponding to 5 benchmarks $\times$ 5 budgets $\times$ 5 masking ratios, RANDOM PSEUDO outperforms the strongest output-based baseline in 89 cells. The number of wins remains high across masking ratios, with 20, 19, 18, 19, and 13 wins out of 25 cells from 10% to 50% masking, respectively. Even at 50% masking, its average Spearman correlation remains above all output-based baselines. This suggests that randomly removed source representations are often approximable from the remaining accessible sources, especially when the remaining pool still covers similar model behaviors.

Adversarial masking exposes the boundary of pseudo-representations. The flat trend under random masking does not hold when the unavailable sources are selected adversarially. ADVDIST PSEUDO masks the most output-distinct sources first and therefore provides a stricter stress test for mixed source pools. Its degradation is small at 10%–20% masking, becomes visible from 30%, and grows substantially at 50%. Correspondingly, the win rate against the strongest output-based baseline drops from 19/25 cells at 10% masking to 13/25 at 30% and 4/25 at 50%. MAE shows the same pattern: from 30% masking onward, ADVDIST PSEUDO has higher MAE than RANDOM PSEUDO in all 25 per-benchmark $\times$ per-budget cells. This suggests that pseudo-representations are less reliable when the masked sources are behaviorally distinctive, since the item representations are harder to approximate from the remaining accessible sources using output-similarity weights.

Sensitivity varies across benchmarks. The effect of adversarial masking is benchmark-dependent. At 50% masking, the Spearman gap between RANDOM PSEUDO and ADVDIST PSEUDO, averaged across budgets, follows BBH (+.048) > MMLU-Pro (+.033) > SEED-Bench-2-Plus (+.025) > ARC-Challenge (+.022) > GSM8K (+.012). The MAE gap follows a similar trend, with MMLU-Pro (+.008) and BBH (+.008) showing the largest degradation, followed by SEED-Bench-2-Plus (+.007), ARC-Challenge (+.005), and GSM8K (+.003). These patterns suggest that task-heterogeneous benchmarks are more sensitive to the loss of behaviorally distinctive source models, as their broader task coverage makes the remaining accessible sources less likely to provide good approximations across all item groups.

Taken together, these results support a bounded view of mixed source pools: pseudo-representations provide a useful partial-access heuristic under random or moderate missingness, but the approximation quality degrades when the hidden states of behaviorally distinctive sources become unavailable.

Table 28. Mean performance under 10% masking. Spearman ρ ($\uparrow$) and MAE ($\downarrow$) are averaged over 10 source-model combinations $\times$ 50 random anchor selections. For Random Pseudo, scores are further averaged over 10 stochastic masking draws; REPCORE and ADVDIST PSEUDO are deterministic under each source-model combination. **Bold** indicates the best and underline the second best per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	REPCORE	0.7070	<u>0.0720</u>	0.8200	0.0500	0.8710	0.0409	0.9000	0.0358	<u>0.9210</u>	0.0321	0.8438	0.0462
	RANDOM PSEUDO	0.7267	0.0754	0.8358	<u>0.0543</u>	0.8821	<u>0.0449</u>	0.9083	<u>0.0390</u>	0.9241	<u>0.0349</u>	0.8554	0.0497
	ADVIST PSEUDO	<u>0.7466</u>	0.0797	<u>0.8303</u>	<u>0.0567</u>	<u>0.8744</u>	<u>0.0467</u>	<u>0.9018</u>	<u>0.0410</u>	0.9188	<u>0.0376</u>	<u>0.8544</u>	0.0523
	BEST BASELINE	0.7520	0.0717	<u>0.8230</u>	<u>0.0573</u>	<u>0.8640</u>	<u>0.0505</u>	<u>0.8900</u>	<u>0.0432</u>	0.9180	<u>0.0357</u>	<u>0.8494</u>	0.0517
BBH	REPCORE	0.7180	<u>0.0950</u>	<u>0.8240</u>	<u>0.0687</u>	0.8700	0.0569	0.8980	0.0494	0.9130	0.0452	<u>0.8446</u>	0.0630
	RANDOM PSEUDO	<u>0.7313</u>	0.1051	0.8263	<u>0.0742</u>	0.8714	0.0600	<u>0.8963</u>	<u>0.0525</u>	<u>0.9129</u>	<u>0.0476</u>	0.8476	0.0679
	ADVIST PSEUDO	0.7336	0.1051	0.8216	<u>0.0733</u>	0.8613	0.0618	0.8925	<u>0.0532</u>	<u>0.9073</u>	<u>0.0477</u>	<u>0.8433</u>	0.0682
	BEST BASELINE	0.7010	0.0931	0.7990	0.0678	0.8520	<u>0.0583</u>	<u>0.8840</u>	<u>0.0516</u>	0.9050	<u>0.0454</u>	0.8282	<u>0.0632</u>
GSM8K	REPCORE	0.7630	<u>0.0714</u>	0.8720	0.0473	0.9050	0.0393	0.9260	0.0350	0.9380	0.0316	0.8808	0.0449
	RANDOM PSEUDO	<u>0.7593</u>	<u>0.0735</u>	<u>0.8578</u>	<u>0.0556</u>	<u>0.9010</u>	<u>0.0463</u>	<u>0.9230</u>	<u>0.0407</u>	<u>0.9367</u>	<u>0.0369</u>	<u>0.8756</u>	0.0506
	ADVIST PSEUDO	0.7505	<u>0.0738</u>	<u>0.8545</u>	<u>0.0555</u>	0.8944	0.0488	0.9185	<u>0.0434</u>	<u>0.9333</u>	<u>0.0399</u>	0.8702	0.0523
	BEST BASELINE	0.7500	0.0644	0.8390	<u>0.0569</u>	0.8700	0.0488	0.8980	<u>0.0418</u>	<u>0.9170</u>	<u>0.0336</u>	0.8548	<u>0.0491</u>
MMLU-Pro	REPCORE	<u>0.7320</u>	0.0910	0.8290	<u>0.0666</u>	0.8750	<u>0.0548</u>	0.9060	<u>0.0476</u>	0.9240	<u>0.0426</u>	0.8532	<u>0.0605</u>
	RANDOM PSEUDO	0.7140	0.0958	0.8327	<u>0.0692</u>	0.8825	<u>0.0560</u>	<u>0.9078</u>	<u>0.0491</u>	<u>0.9236</u>	<u>0.0441</u>	0.8521	0.0628
	ADVIST PSEUDO	0.7212	<u>0.0936</u>	<u>0.8314</u>	<u>0.0688</u>	<u>0.8811</u>	<u>0.0556</u>	0.9086	<u>0.0491</u>	<u>0.9233</u>	<u>0.0441</u>	<u>0.8531</u>	0.0622
	BEST BASELINE	0.7540	<u>0.0917</u>	0.8120	0.0651	0.8660	0.0528	0.8990	0.0457	<u>0.9200</u>	0.0405	0.8502	0.0592
SEED-Bench-2-Plus	REPCORE	<u>0.6420</u>	<u>0.0835</u>	0.7550	0.0600	0.8190	<u>0.0508</u>	0.8530	<u>0.0446</u>	0.8740	<u>0.0402</u>	<u>0.7886</u>	<u>0.0558</u>
	RANDOM PSEUDO	<u>0.6239</u>	<u>0.0881</u>	0.7504	<u>0.0624</u>	0.8088	<u>0.0514</u>	<u>0.8435</u>	<u>0.0452</u>	<u>0.8682</u>	<u>0.0407</u>	<u>0.7790</u>	<u>0.0576</u>
	ADVIST PSEUDO	0.6302	<u>0.0894</u>	<u>0.7651</u>	<u>0.0621</u>	0.8057	0.0500	0.8370	<u>0.0442</u>	0.8628	<u>0.0399</u>	0.7802	0.0571
	BEST BASELINE	0.6760	0.0810	0.7760	<u>0.0628</u>	<u>0.8130</u>	<u>0.0500</u>	0.8360	0.0428	0.8630	0.0378	0.7928	0.0549
Average	REPCORE	0.7124	<u>0.0826</u>	0.8200	0.0585	<u>0.8680</u>	0.0485	0.8966	0.0425	0.9140	0.0383	0.8422	0.0541
	RANDOM PSEUDO	0.7110	0.0876	0.8206	<u>0.0631</u>	0.8692	<u>0.0517</u>	0.8958	<u>0.0453</u>	<u>0.9131</u>	<u>0.0408</u>	<u>0.8419</u>	0.0577
	ADVIST PSEUDO	<u>0.7164</u>	0.0883	<u>0.8206</u>	<u>0.0633</u>	0.8634	<u>0.0526</u>	0.8917	<u>0.0462</u>	0.9091	<u>0.0418</u>	<u>0.8402</u>	0.0584
	BEST BASELINE	0.7266	0.0804	0.8098	<u>0.0620</u>	0.8530	<u>0.0521</u>	0.8814	<u>0.0450</u>	<u>0.9046</u>	<u>0.0386</u>	0.8351	<u>0.0556</u>

Table 29. Mean performance under 20% masking. Spearman ρ ($\uparrow$) and MAE ($\downarrow$) are averaged over 10 source-model combinations $\times$ 50 random anchor selections. For Random Pseudo, scores are further averaged over 10 stochastic masking draws; REPCORE and ADVDIST PSEUDO are deterministic under each source-model combination. **Bold** indicates the best and underline the second best per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	REPCORE	0.7070	<u>0.0720</u>	0.8200	0.0500	0.8710	0.0409	0.9000	0.0358	<u>0.9210</u>	0.0321	0.8438	0.0462
	RANDOM PSEUDO	0.7288	0.0772	0.8401	<u>0.0557</u>	0.8851	<u>0.0457</u>	0.9103	<u>0.0398</u>	0.9265	<u>0.0357</u>	0.8582	<u>0.0508</u>
	ADVIST PSEUDO	<u>0.7407</u>	0.0843	<u>0.8373</u>	<u>0.0632</u>	<u>0.8797</u>	<u>0.0512</u>	<u>0.9055</u>	<u>0.0448</u>	0.9208	<u>0.0399</u>	<u>0.8568</u>	<u>0.0567</u>
	BEST BASELINE	0.7520	0.0717	<u>0.8230</u>	<u>0.0573</u>	<u>0.8640</u>	<u>0.0505</u>	<u>0.8900</u>	<u>0.0432</u>	0.9180	<u>0.0357</u>	<u>0.8494</u>	<u>0.0517</u>
BBH	REPCORE	<u>0.7180</u>	0.0950	<u>0.8240</u>	<u>0.0687</u>	0.8700	0.0569	0.8980	0.0494	<u>0.9130</u>	0.0452	<u>0.8446</u>	0.0630
	RANDOM PSEUDO	0.7226	0.1054	0.8289	0.0744	0.8704	0.0615	<u>0.8979</u>	0.0538	0.9143	0.0487	0.8468	0.0688
	ADVIST PSEUDO	0.6851	0.1098	0.8231	0.0767	0.8632	0.0639	0.8870	0.0560	0.9084	0.0494	<u>0.8334</u>	<u>0.0712</u>
	BEST BASELINE	0.7010	0.0931	0.7990	0.0678	0.8520	<u>0.0583</u>	<u>0.8840</u>	<u>0.0516</u>	0.9050	<u>0.0454</u>	0.8282	<u>0.0632</u>
GSM8K	REPCORE	<u>0.7630</u>	<u>0.0714</u>	0.8720	0.0473	0.9050	0.0393	0.9260	0.0350	<u>0.9380</u>	0.0316	0.8808	0.0449
	RANDOM PSEUDO	0.7680	0.0724	<u>0.8626</u>	<u>0.0534</u>	<u>0.9029</u>	<u>0.0449</u>	<u>0.9252</u>	<u>0.0397</u>	0.9381	0.0363	<u>0.8794</u>	0.0493
	ADVIST PSEUDO	0.7547	0.0741	0.8607	0.0565	0.9015	0.0486	0.9195	0.0436	0.9337	<u>0.0399</u>	0.8740	0.0525
	BEST BASELINE	0.7500	0.0644	0.8390	0.0569	0.8700	0.0488	0.8980	0.0418	0.9170	<u>0.0336</u>	0.8548	<u>0.0491</u>
MMLU-Pro	REPCORE	<u>0.7320</u>	0.0910	<u>0.8290</u>	<u>0.0666</u>	0.8750	<u>0.0548</u>	0.9060	0.0476	0.9240	<u>0.0426</u>	0.8532	<u>0.0605</u>
	RANDOM PSEUDO	0.7103	0.0973	<u>0.8264</u>	<u>0.0686</u>	<u>0.8759</u>	<u>0.0567</u>	0.9043	0.0491	<u>0.9223</u>	<u>0.0442</u>	0.8478	<u>0.0632</u>
	ADVIST PSEUDO	0.7154	0.0979	0.8301	0.0692	0.8760	0.0567	<u>0.9048</u>	0.0492	0.9210	0.0444	0.8495	0.0635
	BEST BASELINE	0.7540	<u>0.0917</u>	0.8120	0.0651	0.8660	0.0528	0.8990	0.0457	0.9200	0.0405	<u>0.8502</u>	0.0592
SEED-Bench-2-Plus	REPCORE	<u>0.6420</u>	<u>0.0835</u>	<u>0.7550</u>	0.0600	0.8190	0.0508	0.8530	<u>0.0446</u>	0.8740	<u>0.0402</u>	<u>0.7886</u>	<u>0.0558</u>
	RANDOM PSEUDO	<u>0.5965</u>	<u>0.0879</u>	<u>0.7339</u>	0.0622	0.7961	0.0514	0.8370	<u>0.0446</u>	0.8608	<u>0.0403</u>	<u>0.7649</u>	<u>0.0573</u>
	ADVIST PSEUDO	0.5688	0.0856	<u>0.7325</u>	<u>0.0603</u>	<u>0.8163</u>	<u>0.0506</u>	<u>0.8491</u>	0.0452	<u>0.8715</u>	0.0408	0.7676	0.0565
	BEST BASELINE	0.6760	0.0810	0.7760	0.0628	0.8130	0.0500	0.8360	0.0428	0.8630	0.0378	0.7928	0.0549
Average	REPCORE	<u>0.7124</u>	<u>0.0826</u>	0.8200	0.0585	0.8680	0.0485	0.8966	0.0425	0.9140	0.0383	0.8422	0.0541
	RANDOM PSEUDO	<u>0.7052</u>	0.0880	<u>0.8184</u>	0.0629	0.8661	<u>0.0520</u>	0.8949	0.0454	<u>0.9124</u>	0.0410	<u>0.8394</u>	<u>0.0579</u>
	ADVIST PSEUDO	0.6929	0.0903	<u>0.8167</u>	<u>0.0652</u>	<u>0.8673</u>	<u>0.0542</u>	0.8932	0.0478	0.9111	0.0429	<u>0.8363</u>	<u>0.0601</u>
	BEST BASELINE	0.7266	0.0804	0.8098	<u>0.0620</u>	0.8530	0.0521	0.8814	<u>0.0450</u>	0.9046	<u>0.0386</u>	0.8351	<u>0.0556</u>

Table 30. Mean performance under 30% masking. Spearman ρ ($\uparrow$) and MAE ($\downarrow$) averaged over 10 source-model combinations $\times$ 50 random anchor selections. For Random Pseudo, scores are further averaged over 10 stochastic masking draws; REPCORE and ADVDIST PSEUDO are deterministic under each source-model combination. **Bold** indicates the best and underline the second best per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	REPCORE	0.7070	<u>0.0720</u>	0.8200	0.0500	0.8710	0.0409	0.9000	0.0358	0.9210	0.0321	0.8438	0.0462
	RANDOM PSEUDO	<u>0.7238</u>	0.0768	0.8344	<u>0.0545</u>	0.8841	<u>0.0452</u>	0.9102	<u>0.0396</u>	0.9257	<u>0.0355</u>	0.8556	0.0503
	ADVIST PSEUDO	0.7149	0.0800	<u>0.8299</u>	<u>0.0594</u>	<u>0.8806</u>	<u>0.0492</u>	<u>0.9018</u>	<u>0.0433</u>	<u>0.9211</u>	<u>0.0391</u>	<u>0.8497</u>	0.0542
	BEST BASELINE	0.7520	0.0717	<u>0.8230</u>	0.0573	<u>0.8640</u>	0.0505	<u>0.8900</u>	0.0432	<u>0.9180</u>	0.0357	<u>0.8494</u>	0.0517
BBH	REPCORE	<u>0.7180</u>	<u>0.0950</u>	<u>0.8240</u>	<u>0.0687</u>	0.8700	0.0569	0.8980	0.0494	0.9130	0.0452	0.8446	0.0630
	RANDOM PSEUDO	0.7200	0.1051	0.8251	0.0749	<u>0.8666</u>	<u>0.0626</u>	<u>0.8918</u>	0.0547	<u>0.9092</u>	0.0496	<u>0.8425</u>	0.0694
	ADVIST PSEUDO	0.6868	0.1076	0.8068	0.0774	<u>0.8553</u>	<u>0.0638</u>	<u>0.8823</u>	<u>0.0565</u>	<u>0.9014</u>	<u>0.0504</u>	<u>0.8265</u>	0.0711
	BEST BASELINE	0.7010	0.0931	0.7990	0.0678	<u>0.8520</u>	<u>0.0583</u>	<u>0.8840</u>	<u>0.0516</u>	<u>0.9050</u>	<u>0.0454</u>	<u>0.8282</u>	<u>0.0632</u>
GSM8K	REPCORE	<u>0.7630</u>	<u>0.0714</u>	0.8720	0.0473	0.9050	0.0393	0.9260	0.0350	0.9380	0.0316	0.8808	0.0449
	RANDOM PSEUDO	0.7634	0.0747	<u>0.8603</u>	<u>0.0548</u>	<u>0.9003</u>	<u>0.0466</u>	<u>0.9215</u>	<u>0.0411</u>	<u>0.9356</u>	0.0372	<u>0.8762</u>	0.0509
	ADVIST PSEUDO	0.7499	0.0766	<u>0.8534</u>	<u>0.0608</u>	<u>0.8909</u>	<u>0.0517</u>	<u>0.9148</u>	<u>0.0447</u>	<u>0.9329</u>	<u>0.0396</u>	<u>0.8684</u>	0.0547
	BEST BASELINE	0.7500	0.0644	<u>0.8390</u>	<u>0.0569</u>	<u>0.8700</u>	<u>0.0488</u>	<u>0.8980</u>	<u>0.0418</u>	<u>0.9170</u>	<u>0.0336</u>	<u>0.8548</u>	<u>0.0491</u>
MMLU-Pro	REPCORE	<u>0.7320</u>	0.0910	0.8290	<u>0.0666</u>	<u>0.8750</u>	<u>0.0548</u>	0.9060	<u>0.0476</u>	0.9240	<u>0.0426</u>	0.8532	<u>0.0605</u>
	RANDOM PSEUDO	0.7124	0.0975	<u>0.8250</u>	<u>0.0688</u>	0.8760	<u>0.0568</u>	<u>0.9034</u>	<u>0.0495</u>	<u>0.9210</u>	<u>0.0445</u>	<u>0.8476</u>	<u>0.0634</u>
	ADVIST PSEUDO	0.6828	0.0995	<u>0.8130</u>	<u>0.0711</u>	<u>0.8646</u>	<u>0.0593</u>	<u>0.8944</u>	<u>0.0523</u>	<u>0.9108</u>	<u>0.0474</u>	<u>0.8331</u>	<u>0.0659</u>
	BEST BASELINE	0.7540	<u>0.0917</u>	<u>0.8120</u>	0.0651	<u>0.8660</u>	0.0528	<u>0.8990</u>	0.0457	<u>0.9200</u>	0.0405	<u>0.8502</u>	0.0592
SEED-Bench-2-Plus	REPCORE	<u>0.6420</u>	<u>0.0835</u>	<u>0.7550</u>	0.0600	0.8190	<u>0.0508</u>	0.8530	<u>0.0446</u>	0.8740	<u>0.0402</u>	<u>0.7886</u>	<u>0.0558</u>
	RANDOM PSEUDO	<u>0.6035</u>	<u>0.0881</u>	<u>0.7308</u>	<u>0.0627</u>	<u>0.7942</u>	<u>0.0519</u>	<u>0.8338</u>	<u>0.0453</u>	<u>0.8613</u>	<u>0.0411</u>	<u>0.7647</u>	<u>0.0578</u>
	ADVIST PSEUDO	0.5848	0.0893	<u>0.7211</u>	<u>0.0648</u>	<u>0.7968</u>	<u>0.0539</u>	<u>0.8419</u>	<u>0.0469</u>	<u>0.8663</u>	<u>0.0425</u>	<u>0.7622</u>	<u>0.0595</u>
	BEST BASELINE	0.6760	0.0810	0.7760	<u>0.0628</u>	<u>0.8130</u>	0.0500	<u>0.8360</u>	0.0428	<u>0.8630</u>	0.0378	0.7928	0.0549
Average	REPCORE	<u>0.7124</u>	<u>0.0826</u>	0.8200	0.0585	0.8680	0.0485	0.8966	0.0425	0.9140	0.0383	0.8422	0.0541
	RANDOM PSEUDO	<u>0.7046</u>	<u>0.0884</u>	<u>0.8151</u>	<u>0.0631</u>	<u>0.8642</u>	<u>0.0526</u>	<u>0.8921</u>	<u>0.0460</u>	<u>0.9106</u>	<u>0.0416</u>	<u>0.8373</u>	<u>0.0584</u>
	ADVIST PSEUDO	0.6838	0.0906	<u>0.8048</u>	<u>0.0667</u>	<u>0.8576</u>	<u>0.0556</u>	<u>0.8870</u>	<u>0.0487</u>	<u>0.9065</u>	<u>0.0438</u>	<u>0.8280</u>	<u>0.0611</u>
	BEST BASELINE	0.7266	0.0804	<u>0.8098</u>	<u>0.0620</u>	<u>0.8530</u>	<u>0.0521</u>	<u>0.8814</u>	<u>0.0450</u>	<u>0.9046</u>	<u>0.0386</u>	<u>0.8351</u>	<u>0.0556</u>

Table 31. Mean performance under 40% masking. Spearman ρ ($\uparrow$) and MAE ($\downarrow$) are averaged over 10 source-model combinations $\times$ 50 random anchor selections. For Random Pseudo, scores are further averaged over 10 stochastic masking draws; REPCORE and ADVDIST PSEUDO are deterministic under each source-model combination. **Bold** indicates the best and underline the second best per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	REPCORE	0.7070	0.0720	0.8200	0.0500	0.8710	0.0409	0.9000	0.0358	0.9210	0.0321	0.8438	0.0462
	RANDOM PSEUDO	0.7243	0.0771	0.8345	0.0548	0.8824	0.0457	0.9085	0.0400	0.9250	0.0360	0.8549	0.0507
	ADVIST PSEUDO	0.7004	0.0810	0.8078	0.0589	0.8486	0.0514	0.8717	0.0477	0.8843	0.0441	0.8226	0.0566
	BEST BASELINE	0.7520	0.0717	0.8230	0.0573	0.8640	0.0505	0.8900	0.0432	0.9180	0.0357	0.8494	0.0517
BBH	REPCORE	0.7180	0.0950	0.8240	0.0687	0.8700	0.0569	0.8980	0.0494	0.9130	0.0452	0.8446	0.0630
	RANDOM PSEUDO	0.7094	0.1065	0.8210	0.0752	0.8674	0.0624	0.8936	0.0546	0.9102	0.0494	0.8403	0.0696
	ADVIST PSEUDO	0.6768	0.1149	0.7984	0.0775	0.8460	0.0651	0.8722	0.0571	0.8913	0.0517	0.8169	0.0733
	BEST BASELINE	0.7010	0.0931	0.7990	0.0678	0.8520	0.0583	0.8840	0.0516	0.9050	0.0454	0.8282	0.0632
GSM8K	REPCORE	0.7630	0.0714	0.8720	0.0473	0.9050	0.0393	0.9260	0.0350	0.9380	0.0316	0.8808	0.0449
	RANDOM PSEUDO	0.7593	0.0736	0.8601	0.0546	0.9011	0.0454	0.9225	0.0402	0.9364	0.0365	0.8759	0.0501
	ADVIST PSEUDO	0.7595	0.0785	0.8479	0.0607	0.8894	0.0497	0.9156	0.0431	0.9308	0.0387	0.8686	0.0541
	BEST BASELINE	0.7500	0.0644	0.8390	0.0569	0.8700	0.0488	0.8980	0.0418	0.9170	0.0336	0.8548	0.0491
MMLU-Pro	REPCORE	0.7320	0.0910	0.8290	0.0666	0.8750	0.0548	0.9060	0.0476	0.9240	0.0426	0.8532	0.0605
	RANDOM PSEUDO	0.7034	0.0982	0.8248	0.0691	0.8741	0.0570	0.9026	0.0498	0.9204	0.0447	0.8451	0.0638
	ADVIST PSEUDO	0.6719	0.1006	0.8185	0.0718	0.8632	0.0595	0.8939	0.0527	0.9126	0.0483	0.8320	0.0666
	BEST BASELINE	0.7540	0.0917	0.8120	0.0651	0.8660	0.0528	0.8990	0.0457	0.9200	0.0405	0.8502	0.0592
SEED-Bench-2-Plus	REPCORE	0.6420	0.0835	0.7550	0.0600	0.8190	0.0508	0.8530	0.0446	0.8740	0.0402	0.7886	0.0558
	RANDOM PSEUDO	0.6025	0.0875	0.7328	0.0637	0.7972	0.0520	0.8361	0.0457	0.8629	0.0412	0.7663	0.0580
	ADVIST PSEUDO	0.6061	0.0889	0.7149	0.0648	0.7901	0.0544	0.8325	0.0496	0.8666	0.0454	0.7620	0.0606
	BEST BASELINE	0.6760	0.0810	0.7760	0.0628	0.8130	0.0500	0.8360	0.0428	0.8630	0.0378	0.7928	0.0549
Average	REPCORE	0.7124	0.0826	0.8200	0.0585	0.8680	0.0485	0.8966	0.0425	0.9140	0.0383	0.8422	0.0541
	RANDOM PSEUDO	0.6998	0.0886	0.8146	0.0635	0.8644	0.0525	0.8927	0.0461	0.9110	0.0416	0.8365	0.0584
	ADVIST PSEUDO	0.6829	0.0928	0.7975	0.0667	0.8475	0.0560	0.8772	0.0500	0.8971	0.0456	0.8204	0.0622
	BEST BASELINE	0.7266	0.0804	0.8098	0.0620	0.8530	0.0521	0.8814	0.0450	0.9046	0.0386	0.8351	0.0556

Table 32. Mean performance under 50% masking. Spearman ρ ($\uparrow$) and MAE ($\downarrow$) are averaged over 10 source-model combinations $\times$ 50 random anchor selections. For Random Pseudo, scores are further averaged over 10 stochastic masking draws; REPCORE and ADVDIST PSEUDO are deterministic under each source-model combination. **Bold** indicates the best and underline the second best per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	REPCORE	0.7070	0.0720	0.8200	0.0500	0.8710	0.0409	0.9000	0.0358	0.9210	0.0321	0.8438	0.0462
	RANDOM PSEUDO	0.7133	0.0765	0.8277	0.0550	0.8766	0.0454	0.9041	0.0396	0.9207	0.0355	0.8485	0.0504
	ADVIST PSEUDO	0.6929	0.0806	0.7934	0.0615	0.8515	0.0516	0.8880	0.0454	0.9091	0.0399	0.8270	0.0558
	BEST BASELINE	0.7520	0.0717	0.8230	0.0573	0.8640	0.0505	0.8900	0.0432	0.9180	0.0357	0.8494	0.0517
BBH	REPCORE	0.7180	0.0950	0.8240	0.0687	0.8700	0.0569	0.8980	0.0494	0.9130	0.0452	0.8446	0.0630
	RANDOM PSEUDO	0.6999	0.1073	0.8105	0.0762	0.8578	0.0625	0.8834	0.0548	0.9023	0.0497	0.8308	0.0701
	ADVIST PSEUDO	0.6528	0.1121	0.7507	0.0845	0.8067	0.0713	0.8423	0.0628	0.8616	0.0576	0.7828	0.0777
	BEST BASELINE	0.7010	0.0931	0.7990	0.0678	0.8520	0.0583	0.8840	0.0516	0.9050	0.0454	0.8282	0.0632
GSM8K	REPCORE	0.7630	0.0714	0.8720	0.0473	0.9050	0.0393	0.9260	0.0350	0.9380	0.0316	0.8808	0.0449
	RANDOM PSEUDO	0.7629	0.0744	0.8604	0.0550	0.9006	0.0459	0.9215	0.0403	0.9358	0.0367	0.8762	0.0505
	ADVIST PSEUDO	0.7420	0.0755	0.8441	0.0604	0.8903	0.0499	0.9168	0.0430	0.9301	0.0383	0.8647	0.0534
	BEST BASELINE	0.7500	0.0644	0.8390	0.0569	0.8700	0.0488	0.8980	0.0418	0.9170	0.0336	0.8548	0.0491
MMLU-Pro	REPCORE	0.7320	0.0910	0.8290	0.0666	0.8750	0.0548	0.9060	0.0476	0.9240	0.0426	0.8532	0.0605
	RANDOM PSEUDO	0.7059	0.0988	0.8162	0.0700	0.8706	0.0576	0.8990	0.0504	0.9170	0.0456	0.8417	0.0645
	ADVIST PSEUDO	0.6485	0.1050	0.7843	0.0800	0.8378	0.0662	0.8759	0.0587	0.8980	0.0527	0.8089	0.0725
	BEST BASELINE	0.7540	0.0917	0.8120	0.0651	0.8660	0.0528	0.8990	0.0457	0.9200	0.0405	0.8502	0.0592
SEED-Bench-2-Plus	REPCORE	0.6420	0.0835	0.7550	0.0600	0.8190	0.0508	0.8530	0.0446	0.8740	0.0402	0.7886	0.0558
	RANDOM PSEUDO	0.6155	0.0876	0.7343	0.0636	0.7932	0.0531	0.8320	0.0465	0.8579	0.0420	0.7666	0.0586
	ADVIST PSEUDO	0.5642	0.0912	0.7153	0.0718	0.7776	0.0608	0.8144	0.0550	0.8361	0.0506	0.7415	0.0659
	BEST BASELINE	0.6760	0.0810	0.7760	0.0628	0.8130	0.0500	0.8360	0.0428	0.8630	0.0378	0.7928	0.0549
Average	REPCORE	0.7124	0.0826	0.8200	0.0585	0.8680	0.0485	0.8966	0.0425	0.9140	0.0383	0.8422	0.0541
	RANDOM PSEUDO	0.6995	0.0889	0.8098	0.0640	0.8598	0.0529	0.8880	0.0463	0.9067	0.0419	0.8328	0.0588
	ADVIST PSEUDO	0.6601	0.0929	0.7776	0.0716	0.8328	0.0600	0.8675	0.0530	0.8870	0.0478	0.8050	0.0651
	BEST BASELINE	0.7266	0.0804	0.8098	0.0620	0.8530	0.0521	0.8814	0.0450	0.9046	0.0386	0.8351	0.0556

Table 33. *Combo_{std}* under 10% masking: standard deviation across 10 source-model combinations. For Random Pseudo, each combo score is first averaged over 10 stochastic masking draws (i.e., an additional *Draw_{std}* averaging layer is applied before computing *Combo_{std}*), which structurally reduces the resulting variance, so its values are shown as reference only. REPCORE (0% masking) and ADVDIST PSEUDO are deterministic under each source-model combination, so their comparison is fair. **Bold** marks the lower (better) value per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	RANDOM PSEUDO	0.0193	0.0043	0.0105	0.0038	0.0080	0.0032	0.0057	0.0028	0.0057	0.0024	0.0098	0.0033
	REPCORE	0.0461	0.0049	0.0229	0.0023	0.0197	0.0017	0.0145	0.0015	0.0094	0.0014	0.0225	0.0024
	ADVDIST PSEUDO	0.0233	0.0089	0.0189	0.0059	0.0167	0.0038	0.0133	0.0047	0.0138	0.0047	0.0172	0.0056
BBH	RANDOM PSEUDO	0.0148	0.0043	0.0089	0.0022	0.0063	0.0018	0.0051	0.0015	0.0048	0.0014	0.0080	0.0022
	REPCORE	0.0357	0.0074	0.0290	0.0048	0.0151	0.0020	0.0087	0.0014	0.0057	0.0021	0.0188	0.0035
	ADVDIST PSEUDO	0.0513	0.0088	0.0276	0.0034	0.0223	0.0043	0.0143	0.0030	0.0110	0.0023	0.0253	0.0044
GSM8K	RANDOM PSEUDO	0.0153	0.0061	0.0080	0.0061	0.0055	0.0043	0.0034	0.0034	0.0022	0.0028	0.0069	0.0045
	REPCORE	0.0377	0.0051	0.0106	0.0017	0.0095	0.0020	0.0065	0.0022	0.0060	0.0025	0.0141	0.0027
	ADVDIST PSEUDO	0.0230	0.0122	0.0202	0.0081	0.0181	0.0047	0.0116	0.0046	0.0085	0.0047	0.0163	0.0069
MMLU-Pro	RANDOM PSEUDO	0.0197	0.0033	0.0090	0.0010	0.0044	0.0010	0.0045	0.0007	0.0036	0.0007	0.0082	0.0013
	REPCORE	0.0341	0.0052	0.0183	0.0031	0.0121	0.0021	0.0061	0.0010	0.0072	0.0017	0.0156	0.0026
	ADVDIST PSEUDO	0.0456	0.0057	0.0219	0.0017	0.0123	0.0024	0.0101	0.0018	0.0067	0.0015	0.0193	0.0026
SEED-Bench -2-Plus	RANDOM PSEUDO	0.0320	0.0023	0.0205	0.0017	0.0159	0.0018	0.0164	0.0019	0.0125	0.0017	0.0195	0.0019
	REPCORE	0.0387	0.0106	0.0503	0.0037	0.0356	0.0018	0.0212	0.0022	0.0192	0.0018	0.0330	0.0040
	ADVDIST PSEUDO	0.0424	0.0089	0.0324	0.0041	0.0183	0.0030	0.0172	0.0028	0.0154	0.0029	0.0251	0.0043
Average	RANDOM PSEUDO	0.0202	0.0041	0.0114	0.0030	0.0080	0.0024	0.0070	0.0021	0.0058	0.0018	0.0105	0.0027
	REPCORE	0.0385	0.0066	0.0262	0.0031	0.0184	0.0019	0.0114	0.0017	0.0095	0.0019	0.0208	0.0030
	ADVDIST PSEUDO	0.0371	0.0089	0.0242	0.0046	0.0175	0.0036	0.0133	0.0034	0.0111	0.0032	0.0206	0.0048

Table 34. *Combo_{std}* under 20% masking: standard deviation across 10 source-model combinations. For Random Pseudo, each combo score is first averaged over 10 stochastic masking draws (i.e., an additional *Draw_{std}* averaging layer is applied before computing *Combo_{std}*), which structurally reduces the resulting variance, so its values are shown as reference only. REPCORE (0% masking) and ADVDIST PSEUDO are deterministic under each source-model combination, so their comparison is fair. **Bold** marks the lower (better) value per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	RANDOM PSEUDO	0.0193	0.0032	0.0085	0.0028	0.0051	0.0022	0.0040	0.0022	0.0027	0.0020	0.0079	0.0025
	REPCORE	0.0461	0.0049	0.0229	0.0023	0.0197	0.0017	0.0145	0.0015	0.0094	0.0014	0.0225	0.0024
	ADVDIST PSEUDO	0.0388	0.0064	0.0173	0.0091	0.0112	0.0057	0.0066	0.0041	0.0064	0.0033	0.0161	0.0057
BBH	RANDOM PSEUDO	0.0170	0.0032	0.0062	0.0019	0.0080	0.0012	0.0070	0.0015	0.0054	0.0017	0.0087	0.0019
	REPCORE	0.0357	0.0074	0.0290	0.0048	0.0151	0.0020	0.0087	0.0014	0.0057	0.0021	0.0188	0.0035
	ADVDIST PSEUDO	0.0563	0.0055	0.0280	0.0070	0.0235	0.0047	0.0152	0.0032	0.0080	0.0030	0.0262	0.0047
GSM8K	RANDOM PSEUDO	0.0167	0.0030	0.0068	0.0021	0.0033	0.0023	0.0025	0.0023	0.0029	0.0021	0.0064	0.0024
	REPCORE	0.0377	0.0051	0.0106	0.0017	0.0095	0.0020	0.0065	0.0022	0.0060	0.0025	0.0141	0.0027
	ADVDIST PSEUDO	0.0302	0.0089	0.0193	0.0074	0.0107	0.0054	0.0084	0.0046	0.0046	0.0053	0.0146	0.0063
MMLU-Pro	RANDOM PSEUDO	0.0199	0.0023	0.0059	0.0009	0.0044	0.0006	0.0022	0.0005	0.0029	0.0005	0.0071	0.0010
	REPCORE	0.0341	0.0052	0.0183	0.0031	0.0121	0.0021	0.0061	0.0010	0.0072	0.0017	0.0156	0.0026
	ADVDIST PSEUDO	0.0340	0.0049	0.0235	0.0033	0.0103	0.0020	0.0092	0.0018	0.0079	0.0018	0.0170	0.0028
SEED-Bench -2-Plus	RANDOM PSEUDO	0.0305	0.0025	0.0171	0.0008	0.0146	0.0013	0.0089	0.0010	0.0076	0.0009	0.0157	0.0013
	REPCORE	0.0387	0.0106	0.0503	0.0037	0.0356	0.0018	0.0212	0.0022	0.0192	0.0018	0.0330	0.0040
	ADVDIST PSEUDO	0.0643	0.0074	0.0323	0.0032	0.0232	0.0026	0.0245	0.0033	0.0181	0.0029	0.0325	0.0039
Average	RANDOM PSEUDO	0.0207	0.0028	0.0089	0.0017	0.0071	0.0015	0.0049	0.0015	0.0043	0.0014	0.0092	0.0018
	REPCORE	0.0385	0.0066	0.0262	0.0031	0.0184	0.0019	0.0114	0.0017	0.0095	0.0019	0.0208	0.0030
	ADVDIST PSEUDO	0.0447	0.0066	0.0241	0.0060	0.0158	0.0041	0.0128	0.0034	0.0090	0.0033	0.0213	0.0047

Table 35. *Combo_{std}* under 30% masking: standard deviation across 10 source-model combinations. For Random Pseudo, each combo score is first averaged over 10 stochastic masking draws (i.e., an additional *Draw_{std}* averaging layer is applied before computing *Combo_{std}*), which structurally reduces the resulting variance, so its values are shown as reference only. REPCORE (0% masking) and ADVDIST PSEUDO are deterministic under each source-model combination, so their comparison is fair. **Bold** marks the lower (better) value per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	RANDOM PSEUDO	0.0203	0.0017	0.0100	0.0022	0.0054	0.0023	0.0043	0.0025	0.0032	0.0022	0.0086	0.0022
	REPCORE	0.0461	0.0049	0.0229	0.0023	0.0197	0.0017	0.0145	0.0015	0.0094	0.0014	0.0225	0.0024
	ADVDIST PSEUDO	0.0341	0.0046	0.0243	0.0052	0.0158	0.0056	0.0138	0.0059	0.0106	0.0051	0.0197	0.0053
BBH	RANDOM PSEUDO	0.0105	0.0028	0.0134	0.0016	0.0111	0.0012	0.0099	0.0016	0.0078	0.0017	0.0105	0.0018
	REPCORE	0.0357	0.0074	0.0290	0.0048	0.0151	0.0020	0.0087	0.0014	0.0057	0.0021	0.0188	0.0035
	ADVDIST PSEUDO	0.0733	0.0109	0.0468	0.0050	0.0344	0.0035	0.0227	0.0036	0.0194	0.0033	0.0393	0.0053
GSM8K	RANDOM PSEUDO	0.0127	0.0026	0.0078	0.0020	0.0034	0.0019	0.0022	0.0021	0.0029	0.0018	0.0058	0.0021
	REPCORE	0.0377	0.0051	0.0106	0.0017	0.0095	0.0020	0.0065	0.0022	0.0060	0.0025	0.0141	0.0027
	ADVDIST PSEUDO	0.0336	0.0082	0.0173	0.0087	0.0144	0.0073	0.0116	0.0066	0.0078	0.0059	0.0169	0.0073
MMLU-Pro	RANDOM PSEUDO	0.0125	0.0032	0.0090	0.0009	0.0049	0.0008	0.0033	0.0009	0.0027	0.0006	0.0065	0.0013
	REPCORE	0.0341	0.0052	0.0183	0.0031	0.0121	0.0021	0.0061	0.0010	0.0072	0.0017	0.0156	0.0026
	ADVDIST PSEUDO	0.0573	0.0045	0.0252	0.0056	0.0162	0.0023	0.0105	0.0023	0.0107	0.0021	0.0240	0.0034
SEED-Bench-2-Plus	RANDOM PSEUDO	0.0203	0.0026	0.0185	0.0011	0.0161	0.0012	0.0101	0.0010	0.0076	0.0008	0.0145	0.0013
	REPCORE	0.0387	0.0106	0.0503	0.0037	0.0356	0.0018	0.0212	0.0022	0.0192	0.0018	0.0330	0.0040
	ADVDIST PSEUDO	0.0825	0.0042	0.0388	0.0049	0.0268	0.0056	0.0287	0.0044	0.0219	0.0028	0.0397	0.0044
Average	RANDOM PSEUDO	0.0153	0.0026	0.0117	0.0016	0.0082	0.0015	0.0060	0.0016	0.0048	0.0014	0.0092	0.0017
	REPCORE	0.0385	0.0066	0.0262	0.0031	0.0184	0.0019	0.0114	0.0017	0.0095	0.0019	0.0208	0.0030
	ADVDIST PSEUDO	0.0562	0.0065	0.0305	0.0059	0.0215	0.0049	0.0175	0.0046	0.0141	0.0038	0.0279	0.0051

Table 36. *Combo_{std}* under 40% masking: standard deviation across 10 source-model combinations. For Random Pseudo, each combo score is first averaged over 10 stochastic masking draws (i.e., an additional *Draw_{std}* averaging layer is applied before computing *Combo_{std}*), which structurally reduces the resulting variance, so its values are shown as reference only. REPCORE (0% masking) and ADVDIST PSEUDO are deterministic under each source-model combination, so their comparison is fair. **Bold** marks the lower (better) value per column.

Benchmark	Condition	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	RANDOM PSEUDO	0.0108	0.0024	0.0053	0.0025	0.0056	0.0019	0.0048	0.0017	0.0032	0.0014	0.0059	0.0020
	REPCORE	0.0461	0.0049	0.0229	0.0023	0.0197	0.0017	0.0145	0.0015	0.0094	0.0014	0.0225	0.0024
	ADVDIST PSEUDO	0.0474	0.0074	0.0355	0.0061	0.0746	0.0120	0.0901	0.0162	0.1011	0.0187	0.0697	0.0121
BBH	RANDOM PSEUDO	0.0257	0.0035	0.0107	0.0017	0.0075	0.0015	0.0089	0.0015	0.0064	0.0013	0.0118	0.0019
	REPCORE	0.0357	0.0074	0.0290	0.0048	0.0151	0.0020	0.0087	0.0014	0.0057	0.0021	0.0188	0.0035
	ADVDIST PSEUDO	0.0508	0.0118	0.0375	0.0049	0.0189	0.0039	0.0113	0.0030	0.0128	0.0035	0.0263	0.0054
GSM8K	RANDOM PSEUDO	0.0093	0.0040	0.0055	0.0024	0.0036	0.0019	0.0023	0.0017	0.0018	0.0014	0.0045	0.0023
	REPCORE	0.0377	0.0051	0.0106	0.0017	0.0095	0.0020	0.0065	0.0022	0.0060	0.0025	0.0141	0.0027
	ADVDIST PSEUDO	0.0229	0.0062	0.0167	0.0086	0.0148	0.0069	0.0093	0.0073	0.0079	0.0060	0.0143	0.0070
MMLU-Pro	RANDOM PSEUDO	0.0221	0.0020	0.0096	0.0007	0.0063	0.0007	0.0039	0.0007	0.0029	0.0006	0.0090	0.0009
	REPCORE	0.0341	0.0052	0.0183	0.0031	0.0121	0.0021	0.0061	0.0010	0.0072	0.0017	0.0156	0.0026
	ADVDIST PSEUDO	0.0739	0.0077	0.0209	0.0024	0.0192	0.0021	0.0184	0.0030	0.0152	0.0020	0.0295	0.0034
SEED-Bench-2-Plus	RANDOM PSEUDO	0.0283	0.0029	0.0143	0.0016	0.0065	0.0013	0.0088	0.0013	0.0061	0.0012	0.0128	0.0017
	REPCORE	0.0387	0.0106	0.0503	0.0037	0.0356	0.0018	0.0212	0.0022	0.0192	0.0018	0.0330	0.0040
	ADVDIST PSEUDO	0.0779	0.0073	0.0362	0.0039	0.0309	0.0033	0.0255	0.0032	0.0151	0.0041	0.0371	0.0044
Average	RANDOM PSEUDO	0.0192	0.0030	0.0091	0.0018	0.0059	0.0015	0.0057	0.0014	0.0041	0.0012	0.0088	0.0018
	REPCORE	0.0385	0.0066	0.0262	0.0031	0.0184	0.0019	0.0114	0.0017	0.0095	0.0019	0.0208	0.0030
	ADVDIST PSEUDO	0.0546	0.0081	0.0294	0.0052	0.0317	0.0056	0.0309	0.0065	0.0304	0.0069	0.0354	0.0065

Table 37. *Combo_{std}* **under 50% masking**: standard deviation across 10 source-model combinations. For **Random Pseudo**, each combo score is first averaged over 10 stochastic masking draws (i.e., an additional *Draw_{std}* averaging layer is applied before computing *Combo_{std}*), which structurally reduces the resulting variance, so its values are shown as reference only. REPCORE (0% masking) and ADVDIST PSEUDO are deterministic under each source-model combination, so their comparison is fair. **Bold** marks the lower (better) value per column.

Benchmark	Condition	<i>K</i> = 10		<i>K</i> = 20		<i>K</i> = 30		<i>K</i> = 40		<i>K</i> = 50		Avg	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	RANDOM PSEUDO	0.0156	0.0032	0.0131	0.0012	0.0098	0.0011	0.0087	0.0009	0.0081	0.0009	0.0111	0.0015
	REPCORE	0.0461	0.0049	0.0229	0.0023	0.0197	0.0017	0.0145	0.0015	0.0094	0.0014	0.0225	0.0024
	ADVDIST PSEUDO	0.0681	0.0050	0.0400	0.0091	0.0300	0.0062	0.0207	0.0035	0.0155	0.0026	0.0349	0.0053
BBH	RANDOM PSEUDO	0.0307	0.0038	0.0096	0.0015	0.0091	0.0010	0.0058	0.0012	0.0062	0.0015	0.0123	0.0018
	REPCORE	0.0357	0.0074	0.0290	0.0048	0.0151	0.0020	0.0087	0.0014	0.0057	0.0021	0.0188	0.0035
	ADVDIST PSEUDO	0.0633	0.0083	0.0316	0.0060	0.0353	0.0049	0.0220	0.0038	0.0235	0.0040	0.0351	0.0054
GSM8K	RANDOM PSEUDO	0.0116	0.0028	0.0075	0.0027	0.0053	0.0022	0.0038	0.0018	0.0025	0.0019	0.0061	0.0023
	REPCORE	0.0377	0.0051	0.0106	0.0017	0.0095	0.0020	0.0065	0.0022	0.0060	0.0025	0.0141	0.0027
	ADVDIST PSEUDO	0.0278	0.0125	0.0125	0.0059	0.0151	0.0041	0.0088	0.0049	0.0054	0.0030	0.0139	0.0061
MMLU-Pro	RANDOM PSEUDO	0.0171	0.0020	0.0087	0.0010	0.0079	0.0008	0.0064	0.0009	0.0062	0.0010	0.0093	0.0011
	REPCORE	0.0341	0.0052	0.0183	0.0031	0.0121	0.0021	0.0061	0.0010	0.0072	0.0017	0.0156	0.0026
	ADVDIST PSEUDO	0.0704	0.0115	0.0330	0.0065	0.0228	0.0059	0.0277	0.0046	0.0261	0.0039	0.0360	0.0065
SEED-Bench-2-Plus	RANDOM PSEUDO	0.0253	0.0031	0.0126	0.0013	0.0169	0.0012	0.0145	0.0011	0.0106	0.0011	0.0160	0.0016
	REPCORE	0.0387	0.0106	0.0503	0.0037	0.0356	0.0018	0.0212	0.0022	0.0192	0.0018	0.0330	0.0040
	ADVDIST PSEUDO	0.0577	0.0092	0.0337	0.0066	0.0390	0.0087	0.0279	0.0074	0.0215	0.0074	0.0360	0.0079
Average	RANDOM PSEUDO	0.0201	0.0030	0.0103	0.0015	0.0098	0.0013	0.0078	0.0012	0.0067	0.0013	0.0109	0.0016
	REPCORE	0.0385	0.0066	0.0262	0.0031	0.0184	0.0019	0.0114	0.0017	0.0095	0.0019	0.0208	0.0030
	ADVDIST PSEUDO	0.0575	0.0093	0.0302	0.0068	0.0284	0.0060	0.0214	0.0048	0.0184	0.0042	0.0312	0.0062

Table 38. $Draw_{std}$ (**Random Pseudo-label Masking**): standard deviation of Spearman ρ and MAE across 10 random masking draws, measuring sensitivity to which source models are masked. Only random masking is stochastic; REPCORE (0% masking) and adversarial-distance masking are deterministic under each source-model combination.

Benchmark	Ratio	$K = 10$		$K = 20$		$K = 30$		$K = 40$		$K = 50$	
		ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE	ρ	MAE
ARC-Challenge	10%	.0324	.0049	.0207	.0032	.0134	.0033	.0094	.0030	.0080	.0026
	20%	.0404	.0062	.0191	.0050	.0118	.0038	.0087	.0033	.0060	.0030
	30%	.0374	.0072	.0226	.0040	.0139	.0038	.0095	.0037	.0076	.0033
	40%	.0426	.0062	.0210	.0053	.0139	.0049	.0105	.0045	.0092	.0042
	50%	.0478	.0069	.0295	.0044	.0214	.0038	.0160	.0036	.0132	.0033
BBH	10%	.0464	.0064	.0257	.0040	.0164	.0027	.0123	.0022	.0099	.0021
	20%	.0563	.0069	.0275	.0040	.0198	.0034	.0132	.0031	.0117	.0028
	30%	.0509	.0077	.0268	.0048	.0183	.0042	.0149	.0040	.0122	.0039
	40%	.0593	.0088	.0299	.0057	.0180	.0043	.0154	.0044	.0121	.0039
	50%	.0640	.0104	.0358	.0062	.0228	.0047	.0170	.0044	.0149	.0045
GSM8K	10%	.0267	.0078	.0156	.0061	.0095	.0048	.0069	.0041	.0058	.0038
	20%	.0324	.0071	.0156	.0051	.0095	.0045	.0068	.0041	.0063	.0040
	30%	.0315	.0087	.0135	.0058	.0100	.0053	.0085	.0047	.0070	.0046
	40%	.0377	.0077	.0160	.0061	.0091	.0052	.0077	.0045	.0066	.0042
	50%	.0371	.0089	.0156	.0062	.0108	.0052	.0092	.0047	.0069	.0046
MMLU-Pro	10%	.0331	.0043	.0177	.0026	.0110	.0017	.0071	.0014	.0055	.0015
	20%	.0587	.0063	.0209	.0028	.0150	.0021	.0100	.0018	.0077	.0016
	30%	.0486	.0074	.0244	.0033	.0150	.0023	.0099	.0020	.0068	.0018
	40%	.0468	.0060	.0221	.0029	.0147	.0023	.0100	.0022	.0086	.0021
	50%	.0516	.0064	.0263	.0034	.0157	.0027	.0134	.0026	.0105	.0027
SEED-Bench -2-Plus	10%	.0550	.0067	.0342	.0033	.0213	.0023	.0199	.0022	.0178	.0023
	20%	.0833	.0073	.0423	.0031	.0317	.0026	.0220	.0025	.0186	.0023
	30%	.0733	.0071	.0430	.0040	.0333	.0031	.0255	.0027	.0188	.0026
	40%	.0672	.0068	.0428	.0038	.0254	.0033	.0213	.0031	.0190	.0030
	50%	.0645	.0075	.0398	.0042	.0303	.0036	.0232	.0035	.0203	.0034

D. Model Pools for Each Benchmark

This section provides the full model pools used for each benchmark in our experiments. The corresponding model lists are reported in Tables 39–43.

Table 39. Models used for ARC Challenge benchmark.

Benchmark	Model Names
ARC Challenge	qwen2.5-72b-instruct, phi-4, qwen2.5-32b-instruct, llama-3.1-nemotron-70b-instruct-hf, llama-3.3-70b-instruct, qwen2.5-32b, deepseek-r1-distill-qwen-14b, qwen2-72b-instruct, qwen2.5-coder-32b-instruct, qwen2.5-14b-instruct-1m, dracarys-72b-instruct, qwen2.5-14b-instruct, qwen2.5-72b, aceinstruct-72b, smaug-llama-3-70b-instruct-32k, qwq-32b-preview, acemath-72b-instruct, qwen2.5-math-72b-instruct, yi-1.5-34b-chat, phi-3-medium-4k-instruct, yi-1.5-34b-chat-16k, dolphin-2.9.2-qwen2-72b, qwen2-math-72b-instruct, qwen2.5-14b, phi-3-medium-128k-instruct, qwen2.5-7b-instruct, qwen2.5-coder-14b-instruct, llama-3.1-tulu-3-70b-dpo, solar-pro-preview-instruct, hermes-3-llama-3.1-70b, llama-3.1-tulu-3-70b, aya-expanse-32b, internlm2.5-20b-chat, aceinstruct-7b, llama-3.1-tulu-3-70b-sft, internlm2.5-7b-chat, phi-3.5-mini-instruct, dolphin-2.9.2-phi-3-medium-abliterated, mistral-small-instruct-2409, deephermes-3-mistral-24b-preview, dolphin-2.9.1-yi-1.5-34b, falcon3-7b-instruct, yi-1.5-9b-chat, llama-3.1-8b-instruct, yi-1.5-9b, dolphin3.0-r1-mistral-24b, yi-1.5-9b-chat-16k, dolphin-2.9.3-yi-1.5-34b-32k, qwen2.5-coder-7b-instruct, falcon3-10b-instruct, qwen2.5-3b-instruct, openchat-3.6-8b-20240522, qwen2-7b-instruct, dolphin-2.9.1-yi-1.5-9b, ministral-8b-instruct-2410, mistral-nemo-instruct-2407, acemath-7b-instruct, granite-3.1-8b-instruct, dolphin-2.9.3-mistral-nemo-12b, c4ai-command-r7b-12-2024, daredevil-8b-abliterated, llama-3-8b-sfr-iterative-dpo-r, meta-llama-3.1-8b-instruct-abliterated, aya-expanse-8b, neuraldaredevil-8b-abliterated, hermes-3-llama-3.1-8b, granite-3.0-8b-instruct, granite-3.2-8b-instruct, qwen2.5-7b, llama-3.1-tulu-3-8b-dpo, aya-23-35b, llama-3.2-3b-instruct, llama-3.1-tulu-3-8b, yi-1.5-6b-chat, olmo-2-1124-7b-instruct, llama-3.1-tulu-3-8b-sft, mistral-7b-instruct-v0.3, dolphin-2.9.3-mistral-7b-32k, llama-3.1-8b-magpie-ultra, llama-3-refueled, dolphin3.0-llama3.1-8b, yi-1.5-34b, qwen2.5-math-7b-instruct, falcon-11b, yi-1.5-6b, falcon3-3b-instruct, yi-1.5-34b-32k, granite-3.2-2b-instruct, granite-3.1-2b-instruct, hermes-3-llama-3.2-3b, yi-1.5-9b-32k, codestral-22b-v0.1, aya-23-8b, granite-3.0-2b-instruct, qwen2-math-7b, openmath2-llama3.1-8b

Table 40. Models used for BBH benchmark.

Benchmark	Model Names
BBH	phi-4, qwen2.5-72b-instruct, qwq-32b-preview, llama-3.3-70b-instruct, deepseek-r1-distill-llama-70b, llama-3.1-70b-instruct, qwen2.5-14b-instruct, qwen2.5-32b-instruct, llama-3.1-nemotron-70b-instruct-hf, acemath-72b-instruct, qwen2.5-coder-32b-instruct, deepseek-r1-distill-qwen-14b, deepseek-r1-distill-qwen-32b, qwen2.5-72b, smaug-llama-3-70b-instruct-32k, qwen2-72b-instruct, aceinstruct-72b, dracarys-72b-instruct, phi-3-medium-4k-instruct, qwen2.5-14b-instruct-1m, internlm2.5-20b-chat, qwen2.5-math-72b-instruct, dolphin-2.9.2-qwen2-72b, mistral-small-instruct-2409, phi-3-medium-128k-instruct, qwen2.5-coder-14b-instruct, dolphin3.0-r1-mistral-24b, qwen2-math-72b-instruct, internlm2.5-7b-chat, qwen2.5-7b-instruct, deephermes-3-mistral-24b-preview, gemma-2-27b-it, llama-3.1-tulu-3-70b-dpo, llama-3.1-tulu-3-70b, hermes-3-llama-3.1-70b, dolphin-2.9.1-yi-1.5-34b, yi-1.5-34b-chat, phi-3-small-128k-instruct, aya-expanse-32b, deepseek-r1-distill-qwen-7b, solar-pro-preview-instruct, llama-3.1-tulu-3-70b-sft, dolphin-2.9.2-phi-3-medium-abliterated, dolphin-2.9.3-yi-1.5-34b-32k, aceinstruct-7b, deepseek-r1-distill-llama-8b, llama-3.1-8b-instruct, phi-3-small-8k-instruct, mistral-nemo-instruct-2407, gemma-2-9b-it, qwen2.5-coder-7b-instruct, acemath-7b-instruct, codestral-22b-v0.1, granite-3.2-8b-instruct, granite-3.1-8b-instruct, ministral-8b-instruct-2410, falcon3-10b-instruct, yi-1.5-34b-chat-16k, phi-3.5-mini-instruct, llama-3-refueled, qwen2.5-7b, granite-3.0-8b-instruct, dolphin-2.9.3-mistral-nemo-12b, qwen2.5-14b, yi-1.5-34b-32k, qwen2-7b-instruct, aya-23-35b, yi-1.5-9b-chat, openchat-3.6-8b-20240522, llama-3-8b-sfr-iterative-dpo-r, qwen2.5-3b-instruct, hermes-3-llama-3.1-8b, k2-chat, daredevil-8b-abliterated, neuraldaredevil-8b-abliterated, falcon3-7b-instruct, llama-3.2-3b-instruct, dolphin-2.9.1-yi-1.5-9b, llama-3.1-tulu-3-8b-dpo, llama-3.1-tulu-3-8b, qwen2.5-math-7b-instruct, meta-llama-3.1-8b-instruct-abliterated, llama-3.1-tulu-3-8b-sft, openmath2-llama3.1-8b, llama-3.1-8b-magpie-ultra, yi-1.5-9b-chat-16k, dolphin-2.9.3-mistral-7b-32k, dolphin3.0-llama3.1-8b, yi-1.5-9b, aya-expanse-8b, yi-1.5-6b-chat, granite-3.0-8b-base, granite-3.0-2b-instruct, granite-3.2-2b-instruct, olmo-2-1124-7b-instruct, granite-3.1-8b-base, mistral-7b-instruct-v0.3, granite-3.1-2b-instruct, falcon3-3b-instruct, hermes-3-llama-3.2-3b, aya-23-8b

Table 41. Models used for GSM8K benchmark.

Benchmark	Model Names
GSM8K	acemath-72b-instruct, qwen2-math-72b-instruct, llama-3.3-70b-instruct, qwen3-30b-a3b, qwen2.5-72b-instruct, phi-4, qwq-32b-preview, qwen2.5-math-72b-instruct, qwen2.5-32b-instruct, llama-3.1-tulu-3-70b, llama-3.1-70b-instruct, qwen2.5-coder-32b-instruct, deepseek-r1-distill-qwen-14b, llama-3.1-tulu-3-70b-dpo, aceinstruct-72b, qwen2.5-14b-instruct, qwen2-72b-instruct, dracarys-72b-instruct, qwen2.5-14b-instruct-1m, hermes-3-llama-3.1-70b, smaug-llama-3-70b-instruct-32k, qwen2.5-72b, dolphin-2.9.2-qwen2-72b, qwen2.5-7b-instruct, deephermes-3-mistral-24b-preview, llama-3.1-tulu-3-70b-sft, mistral-small-instruct-2409, openmath2-llama3.1-8b, qwen2.5-32b, aceinstruct-7b, phi-3-medium-4k-instruct, llama-3.1-nemotron-70b-instruct-hf, yi-1.5-34b-chat, falcon3-7b-instruct, qwen2.5-14b, dolphin-2.9.2-phi-3-medium-abliterated, phi-3-medium-128k-instruct, deepseek-r1-distill-qwen-7b, gemma-2-9b-it, qwen2.5-coder-7b-instruct, llama-3.1-tulu-3-8b-dpo, ministral-8b-instruct-2410, aya-expanse-32b, dolphin-2.9.1-yi-1.5-34b, qwen2.5-7b, internlm2.5-20b-chat, phi-3.5-mini-instruct, llama-3.1-tulu-3-8b, acemath-7b-instruct, yi-1.5-9b-chat, llama-3.1-8b-instruct, deepseek-r1-distill-qwen-32b, granite-3.2-8b-instruct, mistral-nemo-instruct-2407, qwen2.5-math-7b, daredevil-8b-abliterated, qwen2.5-3b-instruct, dolphin-2.9.3-yi-1.5-34b-32k, dolphin-2.9.3-mistral-nemo-12b, neuraldaredevil-8b-abliterated, granite-3.1-8b-instruct, yi-1.5-9b-chat-16k, internlm2.5-7b-chat, hermes-3-llama-3.1-8b, olmo-2-1124-7b-instruct, qwen2-7b-instruct, llama-3.2-3b-instruct, openchat-3.6-8b-20240522, qwen2-math-7b, dolphin-2.9.1-yi-1.5-9b, deepseek-r1-distill-llama-70b, aya-expanse-8b, llama-3.1-tulu-3-8b-sft, dolphin-2.9.3-mistral-7b-32k, meta-llama-3.1-8b-instruct-abliterated, granite-3.0-8b-instruct, codestral-22b-v0.1, yi-1.5-6b-chat, falcon3-10b-instruct, granite-3.0-8b-base, granite-3.2-2b-instruct, granite-3.1-2b-instruct, llama-3-refueled, granite-3.0-2b-instruct, hermes-3-llama-3.2-3b, granite-3.1-8b-base, aya-23-35b, falcon3-3b-instruct, mistral-7b-instruct-v0.3, deepseek-r1-distill-llama-8b

Table 42. Models used for MMLU-Pro benchmark.

Benchmark	Model Names
MMLU-Pro	qwq-32b-preview, qwen2.5-72b-instruct, llama-3.3-70b-instruct, qwen2.5-32b-instruct, deepseek-r1-distill-qwen-14b, llama-3.1-nemotron-70b-instruct-hf, llama-3.1-70b-instruct, aceinstruct-72b, llama-3.1-tulu-3-70b-dpo, dracarys-72b-instruct, llama-3.1-tulu-3-70b, qwen2.5-14b-instruct, qwen2.5-14b-instruct-1m, qwen2.5-72b, qwen2.5-coder-32b-instruct, acemath-72b-instruct, qwen2.5-math-72b-instruct, qwen2.5-32b, solar-pro-preview-instruct, dolphin-2.9.2-qwen2-72b, smaug-llama-3-70b-instruct-32k, qwen2-math-72b-instruct, hermes-3-llama-3.1-70b, phi-3-medium-4k-instruct, qwen2.5-7b-instruct, deephermes-3-mistral-24b-preview, qwen2.5-coder-14b-instruct, phi-3-medium-128k-instruct, phi-3-small-128k-instruct, gemma-2-27b-it, llama-3.1-tulu-3-70b-sft, mistral-small-instruct-2409, aceinstruct-7b, dolphin-2.9.2-phi-3-medium-abliterated, phi-3-small-8k-instruct, yi-1.5-34b-chat, qwen2.5-14b, yi-1.5-34b-chat-16k, qwen2.5-7b, falcon3-7b-instruct, acemath-7b-instruct, gemma-2-9b-it, phi-3.5-mini-instruct, dolphin-2.9.1-yi-1.5-34b, aya-expanse-32b, qwen2.5-coder-7b-instruct, yi-1.5-9b-chat, qwen2-7b-instruct, mistral-nemo-instruct-2407, llama-3.1-tulu-3-8b-dpo, qwen2.5-3b-instruct, falcon3-10b-instruct, dolphin-2.9.3-yi-1.5-34b-32k, yi-1.5-9b-chat-16k, daredevil-8b-abliterated, llama-3.1-tulu-3-8b, ministral-8b-instruct-2410, dolphin-2.9.1-yi-1.5-9b, neuraldaredevil-8b-abliterated, llama-3.1-70b, yi-1.5-34b-32k, qwen2.5-math-7b-instruct, granite-3.2-8b-instruct, meta-llama-3.1-8b-instruct-abliterated, dolphin-2.9.3-mistral-nemo-12b, k2-chat, falcon3-3b-instruct, hermes-3-llama-3.1-8b, llama-3.1-tulu-3-8b-sft, llama-3-8b-sfr-iterative-dpo-r, yi-1.5-9b, yi-1.5-6b-chat, qwen2.5-math-7b, codestral-22b-v0.1, openchat-3.6-8b-20240522, dolphin3.0-llama3.1-8b, granite-3.0-8b-instruct, olmo-2-1124-7b-instruct, llama-3.1-8b-magpie-ultra, aya-23-35b, aya-expanse-8b, llama-3-refueled, mistral-7b-instruct-v0.3, qwen2-math-7b, granite-3.2-2b-instruct, granite-3.1-2b-instruct, yi-1.5-9b-32k, yi-1.5-6b, dolphin-2.9.3-mistral-7b-32k, granite-3.0-2b-instruct, hermes-3-llama-3.2-3b

Table 43. Models used for SEED-Bench-2-plus benchmark.

Benchmark	Model Names
SEED-Bench-2-Plus	qwen2.5-vl-32b-instruct, internvl2.5-78b-mpo, qwen2.5-vl-72b-instruct, internvl3-78b, internvl3-38b, internvl2.5-38b-mpo, qwen2-vl-72b-instruct, ovis2-16b, ovis2-34b, mimo-vl-7b-rl, internvl2.5-78b, vARCo-vision-2.0-14b, internvl3-14b, internvl2.5-26b-mpo, wethink-qwen2.5vl-7b, internvl2.5-38b, internvl2-llama3-76b, ovis2-8b, internvl3-9b, qwen2.5-vl-7b-instruct, internvl2.5-26b, internvl2.5-8b-mpo, ovis-u1-3b, ovis2-4b, internvl2-40b, internvl3-8b, internvl2.5-8b, sail-vl-1d6-8b, ovis1.6-gemma2-9b, gemma-3-27b-it, sail-vl-1d5-8b, internvl2-8b-mpo, internvl2.5-4b-mpo, qwen2-vl-7b-instruct, internvl2-26b, gemma-3-12b-it, ovis2-2b, internvl2.5-4b, internvl-chat-v1-5, ristretto-3b, sail-vl-1d5-2b, points-qwen-2-5-7b-chat, internvl2-8b, qwen2.5-vl-3b-instruct, vARCo-vision-2.0-1.7b, ovis1.5-llama3-8b, llava-onevision-qwen2-7b-si-hf, llava-onevision-qwen2-7b-ov-hf, points-yi-1-5-9b-chat, phi-3-vision-128k-instruct, internvl2-4b, llama-3.2-11b-vision-instruct, xinyuan-vl-2b, qwen2-vl-2b-instruct, internvl3-2b, internvl2.5-2b-mpo, qtunev11.5-2b, phi-3.5-vision-instruct, granite-vision-3.3-2b, gemma-3-4b-it, minicpm-llama3-v-2.5, granite-vision-3.2-2b, internvl2.5-2b, ovis2-1b, vintern-3b-beta, smolvlm-2.2b-instruct, granite-vision-3.1-2b-preview, h2ovl-mississippi-2b, smolvlm-instruct, llama3-llava-next-8b-hf, mini-internvl-chat-4b-v1-5, internvl2.5-1b-mpo, mini-internvl-chat-2b-v1-5, llava-interleave-qwen-7b-hf, llava-interleave-qwen-7b-dpo-hf, molmo-7b-d-0924, internvl2-2b, qtunev11.5-3b, molmo-7b-o-0924, pixtral-12b, llava-onevision-qwen2-0.5b-ov-hf, internvl2.5-1b, internvl2-1b, phi-4-multimodal-instruct, llava-1.5-13b-hf, smolvlm-synthetic, vintern-1b-v2, sharegpt4v-7b-hf, ovis1.6-llama3.2-3b, llava-onevision-qwen2-0.5b-si-hf, llava-1.5-7b-hf, minicpm-v, minicpm-v-2